ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



KỸ THUẬT DỮ LIỆU (CO5173)

BÁO CÁO BÀI TẬP LỚN

MarketPulse 360: Hệ thống Kỹ thuật Dữ liệu cho Phân tích Marketing Đa kênh

---

GVHD:	PGS. TS. Võ Thị Ngọc Châu	
Nhóm:	09	
SVTH:	Trần Trọng Bách	2570779
	Lê Minh Huy	2570411
	Nguyễn Quang Thiện	2670376
	Nguyễn Khánh Nam	2670331

TP. Hồ Chí Minh, Tháng 11 Năm 2026

Mục lục

1 Giới thiệu đề tài **1**

1.1 Bối cảnh và bài toán 1

1.2 Các bên liên quan 2

1.3 Yêu cầu nghiệp vụ 4

2 Dữ liệu và kiến trúc giải pháp **7**

2.1 Nguồn dữ liệu 7

2.2 Đặc trưng 4V 9

2.3 Kiến trúc tổng thể 10

2.4 Mô hình dữ liệu Medallion 11

2.5 Lựa chọn lưu trữ và phương án thay thế 14

3 Công nghệ quản lý và xử lý dữ liệu **16**

3.1 Apache Kafka và Apicurio Registry 16

3.2 Apache Flink 17

3.3 Lakehouse: MinIO, Iceberg, Nessie và Spark 19

3.4 Dagster, dbt và ClickHouse 20

3.5 Ví dụ minh họa 23

4 Ứng dụng Marketing Command Center **26**

4.1 Kiến trúc ứng dụng 26

4.2 Cơ sở dữ liệu vận hành 28

4.3 Giao diện và yêu cầu nghiệp vụ 30

5 Đánh giá thực nghiệm **35**

5.1 Môi trường và độ đo 35

5.2 Tính đúng đắn của dữ liệu 36

5.3 Flink và Kafka Streams 37

5.4 ClickHouse và PostgreSQL 38

5.5 Hiệu quả của ứng dụng 40

6 Thách thức và kết luận **42**
6.1 Ba thách thức vận hành 42
6.2 Kết luận 44
6.3 Hướng phát triển 45

Danh sách hình vẽ

1.1 Bối cảnh hệ thống và luồng giá trị của MarketPulse 360 2

2.1 Kiến trúc tổng thể kết hợp Streaming và Batch 10
2.2 Luồng biến đổi dữ liệu Medallion Bronze → Silver → Gold 12
2.3 Lược đồ quan hệ tầng Bronze 13
2.4 Lược đồ quan hệ tầng Silver 14

3.1 Topology chuỗi toán tử xử lý luồng có trạng thái trên Apache Flink 18
3.2 Đồ thị phụ thuộc các tài sản dữ liệu trên Dagster 20
3.3 Quy trình kiểm chứng bản phát hành cô lập và công bố nguyên tử 21
3.4 Lược đồ quan hệ tầng Gold và 9 mô hình dbt phục vụ trên ClickHouse . . 22

4.1 Kiến trúc tầng phục vụ ứng dụng Marketing Command Center 26
4.2 Lược đồ quan hệ PostgreSQL: marketpulse_app và marketpulse_ops . . . 29
4.3 Lược đồ quan hệ các bảng ClickHouse phục vụ, journey sidecar và kho dữ liệu trực tiếp 30
4.4 Tỷ trọng chi phí và doanh thu gộp được phân bổ theo nền tảng quảng cáo (01/2025 – 08/2026) 31
4.5 Doanh thu gộp đã kiểm chứng theo ngành hàng và theo thiết bị mua hàng 33
4.6 Doanh thu ròng đã kiểm chứng theo 13 tỉnh/thành phố (khách hàng đã xác định vị trí) 33

5.1 Thông lượng theo quy mô và thời gian phục hồi sau sự cố của Flink và Kafka Streams 38
5.2 Độ trễ truy vấn P_{95} ở 20 triệu dòng và thời gian CPU theo quy mô của ClickHouse và PostgreSQL 39

Danh sách bảng

1.1 Ma trận ánh xạ nhóm nhu cầu, yêu cầu nghiệp vụ, bên liên quan và năng lực Kỹ thuật Dữ liệu 6

2.1 Đặc tả các nguồn dữ liệu, định dạng, khóa liên kết và đặc trưng nổi bật . . 8
2.2 Vai trò của các công cụ xử lý và các thành phần lưu trữ 15

3.1 Cấu hình các Kafka topic 17
3.2 Các mô hình dbt và bài kiểm thử chất lượng dữ liệu tự động 23

4.1 Các không gian làm việc của Marketing Command Center 31
4.2 Đối chiếu Live View với file nguồn tại thời điểm phát lại 32
4.3 Truy vết yêu cầu nghiệp vụ tới màn hình, API (tiền tố `/api/v1`) và dữ liệu 34

5.1 Kết quả đối soát tính đúng đắn của dữ liệu 37
5.2 Thông lượng và độ trễ commit của Apache Flink 2.2.1 và Kafka Streams 4.3.1 38
5.3 Độ trễ truy vấn (ms) của ClickHouse 26.8 và PostgreSQL 17.6 39
5.4 Mười tác vụ ra quyết định và kết quả kiểm chứng tự động 40
5.5 Độ trễ phản hồi của các REST API chính (ms) 41

6.1 Ba thách thức vận hành và giải pháp trên từng chặng của đường ống . . . 42

Chương 1

Giới thiệu đề tài

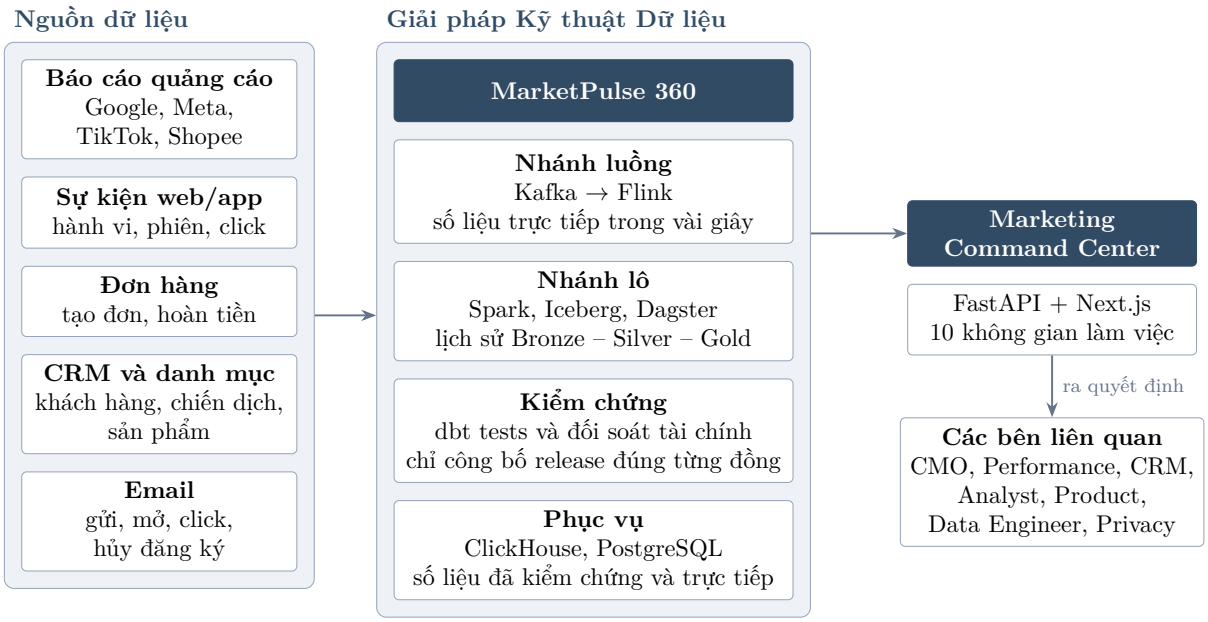
1.1 Bối cảnh và bài toán

Bài toán marketing đa kênh trong bán lẻ

Một doanh nghiệp bán lẻ tại Việt Nam hiện nay hiếm khi chỉ bán hàng qua một kênh. Cùng một lúc, doanh nghiệp chạy nhiều chiến dịch quảng cáo trên nhiều nền tảng khác nhau (Google Ads, Meta Ads, TikTok Ads và Shopee Ads), đưa người dùng về website và ứng dụng di động, ghi nhận đơn hàng và hoàn tiền trên hệ thống thương mại điện tử, quản lý hồ sơ khách hàng trong CRM và nuôi dưỡng khách hàng bằng các chiến dịch email. Mỗi hệ thống trên tự sinh ra báo cáo riêng với định dạng, độ trễ, đơn vị tiền tệ và định danh khác nhau: báo cáo quảng cáo của Google tính chi phí bằng `cost_micros`, của Meta tính bằng chuỗi thập phân, của Shopee ghi ngày theo dạng `DD/MM/YYYY`; sự kiện web đến theo thời gian thực trong khi báo cáo quảng cáo và ảnh chụp CRM đến theo ngày.

Hệ quả là những câu hỏi kinh doanh cơ bản nhất, như *chiến dịch nào thực sự mang lại doanh thu?*, *ngân sách hôm nay có đang bị đốt vào kênh kém hiệu quả không?* hay *khách hàng rời bỏ ở bước nào của phễu?*, lại không thể trả lời bằng bất kỳ báo cáo đơn lẻ nào. Các nền tảng quảng cáo cùng "nhận công" cho một đơn hàng, số liệu đơn hàng không khớp với số liệu chuyển đổi do nền tảng báo cáo, và mỗi phòng ban làm việc với một con số khác nhau.

Đề tài xây dựng **MarketPulse 360**, một sản phẩm dữ liệu marketing đầu-cuối nhằm chứng minh rằng giải pháp Kỹ thuật Dữ liệu có thể vừa *đúng*, vừa *nhanh*, vừa *hữu ích* cho người dùng nghiệp vụ. Hệ thống tiếp nhận năm nhóm nguồn dữ liệu, xử lý song song theo hai nhánh luồng và lô, kiểm chứng tự động trước khi công bố, rồi phục vụ kết quả cho ứng dụng *Marketing Command Center* (Hình 1.1). Toàn bộ mã nguồn, dữ liệu mô phỏng và bằng chứng kiểm thử được lưu trong kho mã của nhóm [1].



Hình 1.1: Bối cảnh hệ thống và luồng giá trị của MarketPulse 360

Hai yêu cầu đối lập: tốc độ và độ chính xác

Bài toán đặt ra hai yêu cầu kéo hệ thống về hai hướng ngược nhau:

- **Tốc độ (độ trễ thấp).** Khi một chiến dịch đột ngột tăng chi phí hoặc tỷ lệ chuyển đổi sụt giảm, người quản lý quảng cáo cần biết trong vài giây đến vài phút chứ không phải sáng hôm sau. Yêu cầu này đòi hỏi xử lý luồng có trạng thái: khử trùng lặp, gom phiên và cập nhật chỉ số chiến dịch ngay khi sự kiện đến.
- **Độ chính xác được kiểm chứng.** Số liệu dùng để phân bổ ngân sách và báo cáo tài chính phải nối đúng hành trình khách hàng trong cửa sổ 30 ngày, loại bỏ phiên bot, tôn trọng lựa chọn đồng thuận (`consent_analytics`, `consent_marketing`) và cân khớp doanh thu ròng đến từng đồng. Những phép tính này cần dữ liệu lịch sử đầy đủ, dữ liệu đến trễ và khả năng tính lại.

Không một công cụ đơn lẻ nào đáp ứng được cả hai. MarketPulse 360 giải quyết bằng cách tách hai loại số liệu một cách tường minh: số liệu *trực tiếp* (tạm thời, cập nhật liên tục) và số liệu *đã kiểm chứng* (chỉ được công bố sau khi vượt qua toàn bộ kiểm thử và đối soát). Hai loại số liệu không bao giờ bị cộng lẫn vào nhau trên giao diện.

1.2 Các bên liên quan

Bảy nhóm người dùng được xác định từ quy trình ra quyết định marketing của một doanh nghiệp bán lẻ. Mỗi nhóm có câu hỏi riêng và cần dữ liệu ở độ chi tiết, độ trễ khác nhau.

Giám đốc Marketing (CMO)

Chịu trách nhiệm phân bổ tổng ngân sách giữa các kênh. CMO cần biết kênh nào tạo ra doanh thu ròng thực sự (sau hoàn tiền), tiến độ giải ngân so với kế hoạch và kết quả dự kiến khi chuyển ngân sách giữa các chiến dịch. Dữ liệu cần ở mức tổng hợp theo ngày, phải được kiểm chứng và có thể giải thích.

Quản lý quảng cáo (Performance Marketing)

Vận hành chiến dịch hằng ngày. Người này cần chỉ số chiến dịch gần thời gian thực (phiên, đơn hàng, doanh thu theo chiến dịch), cảnh báo khi chi phí tăng đột biến hoặc vượt ngân sách, kèm bằng chứng để quyết định tạm dừng hay điều chỉnh giá thầu.

Chuyên viên CRM (Lifecycle Marketing)

Cần hồ sơ khách hàng hợp nhất (Customer 360), phân đoạn theo hành vi mua (RFM), mức độ tương tác email và danh sách khách hàng có cơ hội mua thêm hoặc có nguy cơ rời bỏ. Yêu cầu quan trọng là chỉ được kích hoạt khách hàng đã đồng ý nhận marketing.

Chuyên viên phân tích dữ liệu (Data Analyst)

Cần so sánh các mô hình phân bổ doanh thu (First-Touch, Last-Touch, Linear), phân tích phễu chuyển đổi và cohort giữ chân khách hàng trên dữ liệu đã kiểm chứng, đồng thời truy vết được hành trình của từng đơn hàng khi số liệu bất thường.

Quản lý thương mại điện tử (E-commerce)

Quan tâm đến hiệu quả ngành hàng, thiết bị mua sắm, giá trị đơn hàng, nhịp mua theo khung giờ và vị trí rơi rụng trong phễu từ xem sản phẩm đến thanh toán.

Kỹ sư dữ liệu (Data Engineer)

Chịu trách nhiệm cho tính đúng và tính sẵn sàng của toàn bộ đường ống: theo dõi sức khỏe hệ thống, kết quả kiểm thử chất lượng, nguồn gốc (lineage) của từng bản phát hành, khả năng phát lại và tính lại lịch sử khi công thức thay đổi mà không mất dữ liệu cũ.

Chủ sở hữu dữ liệu (Data Owner)

Bảo đảm dữ liệu cá nhân chỉ được dùng đúng mục đích: lọc theo đồng thuận, ẩn danh hóa định danh trên giao diện, phân quyền theo vai trò và lưu vết kiểm toán mọi truy cập.

1.3 Yêu cầu nghiệp vụ

Nhóm gồm $n = 4$ thành viên nên đề bài yêu cầu tối thiểu $2n = 8$ yêu cầu nghiệp vụ. MarketPulse 360 xác định **10 yêu cầu** (`BR-01` đến `BR-10`), chia thành sáu nhóm nhu cầu. Với mỗi yêu cầu, phần dưới đây nêu rõ dữ liệu cần có và lý do không thể đáp ứng nếu thiếu giải pháp Kỹ thuật Dữ liệu.

Nhóm 1: Chiến dịch mang lại doanh thu (BR-01)

BR-01: So sánh các mô hình phân bổ doanh thu. Phân bổ doanh thu gộp, hoàn tiền và doanh thu ròng cho chiến dịch theo ba mô hình First-Touch, Last-Touch và Linear trên cửa sổ 30 ngày trước đơn hàng, từ đó tính ROAS theo từng mô hình. *Vai trò của Kỹ thuật Dữ liệu:* phải nối click quảng cáo trả phí (trong luồng web) với đơn hàng (trong luồng đơn hàng) của cùng một hồ sơ khách hàng qua nhiều ngày, đồng thời bảo đảm tổng tiền được phân bổ bằng đúng tổng doanh thu, điều mà báo cáo riêng của từng nền tảng quảng cáo không làm được.

Nhóm 2: Phản ứng tức thời (BR-02 – BR-05)

- **BR-02: Giám sát trực tiếp.** Hiển thị chỉ số chiến dịch (phiên theo từng bước phễu, đơn hàng, doanh thu) và lượng truy cập cửa hàng (Live View) trong vài giây sau khi sự kiện xảy ra.
- **BR-03: Cảnh báo có giải thích.** Phát hiện vượt ngân sách, chi phí tăng đột biến hoặc tỷ trọng hoàn tiền bất thường, mỗi cảnh báo kèm bằng chứng (ngưỡng, giá trị quan sát, phiên bản dữ liệu đã dùng).
- **BR-04: Theo dõi tiến độ giải ngân.** So sánh chi phí lũy kế với ngân sách cấu hình cho từng chiến dịch trong kỳ.
- **BR-05: Mô phỏng phân bổ lại ngân sách.** Ước tính doanh thu ròng khi phân bổ lại ngân sách dựa trên ROAS ròng lịch sử, công khai giả định tuyến tính của phép mô phỏng.

Vai trò của Kỹ thuật Dữ liệu: xử lý luồng có trạng thái để có số liệu trong vài giây, nhưng cảnh báo và mô phỏng phải dựa trên số liệu chi phí đã kiểm chứng để không ra quyết định sai từ dữ liệu trùng lặp hoặc đến trễ.

Nhóm 3: Chính xác đến từng đồng (BR-06)

BR-06: Kiểm chứng trước khi công bố. Mọi bản phát hành số liệu phải vượt qua kiểm thử dữ liệu tự động (dbt) và đối soát tài chính trước khi người dùng nhìn thấy; số liệu trực tiếp và số liệu đã kiểm chứng được tách biệt. *Vai trò của Kỹ thuật Dữ liệu:*

cần một quy trình công bố nguyên tử (atomic publication) và sổ cái vận hành để một bản phát hành lỗi không bao giờ thay thế bản đang phục vụ.

Nhóm 4: Hành vi khách hàng (BR-07 – BR-09)

- **BR-07: Phễu chuyển đổi.** Đếm số phiên đạt từng bước `page_view`, `view_item`, `add_to_cart`, `begin_checkout`, `purchase` theo ngày và chiến dịch.
- **BR-08: Hồ sơ Customer 360.** Hồ sơ hợp nhất theo khách hàng, dòng thời gian hoạt động, phân đoạn RFM có lưu phiên bản, mức tương tác email và nhịp mua theo khung giờ.
- **BR-09: Cohort, sản phẩm và địa lý.** Tỷ lệ giữ chân theo cohort tháng, doanh thu theo ngành hàng và thiết bị, phân bố doanh thu theo 13 tỉnh/thành phố.

Vai trò của Kỹ thuật Dữ liệu: hợp nhất định danh (khách đã đăng nhập, khách ẩn danh, phiên) trên nhiều nguồn và nhiều tháng dữ liệu; chỉ một kho dữ liệu lịch sử có quản trị mới tính được cohort và Customer 360 nhất quán.

Nhóm 5 và 6: Quản trị dữ liệu (BR-10)

BR-10: Quản trị dữ liệu. Loại khỏi số liệu đã kiểm chứng mọi sự kiện không có đồng thuận phân tích; chỉ kích hoạt phân đoạn với khách hàng có đồng thuận marketing; ẩn danh hóa định danh trên giao diện; phân quyền ba vai trò `viewer`, `operator`, `admin` kèm nhật ký kiểm toán; và tính lại lịch sử (backfill) một cách cô lập từ các snapshot Bronze đã ghim khi công thức thay đổi. *Vai trò của Kỹ thuật Dữ liệu:* quản trị phải được áp dụng ngay trong đường ống, không thể thêm sau ở tầng báo cáo.

Tổng hợp yêu cầu nghiệp vụ

Bảng 1.1 tổng hợp ánh xạ giữa sáu nhóm nhu cầu, mười yêu cầu nghiệp vụ, các bên liên quan chính và năng lực Kỹ thuật Dữ liệu cần thiết. Chương 4 đối chiếu lại từng yêu cầu với màn hình, API và bảng dữ liệu đã hiện thực.

Bảng 1.1: *Ma trận ánh xạ nhóm nhu cầu, yêu cầu nghiệp vụ, bên liên quan và năng lực Kỹ thuật Dữ liệu*

Nhóm nhu cầu	**Yêu cầu**	**Bên liên quan**	**Năng lực Kỹ thuật Dữ liệu cần có**
1. Chiến dịch nào mang lại doanh thu	BR-01 Phân bổ đa mô hình	CMO và Analyst	Nối click–đơn hàng 30 ngày, phân bổ số nguyên bảo toàn tổng tiền
2. Phản ứng tức thời	BR-02 Giám sát trực tiếp; BR-03 Cảnh báo; BR-04 Giải ngân; BR-05 Mô phỏng	Performance và CMO	Xử lý luồng có trạng thái, khử trùng lặp, chỉ số chi phí đã kiểm chứng
3. Báo cáo đúng từng đồng	BR-06 Kiểm chứng trước công bố	Data Engineer và CMO	dbt tests, đối soát tài chính, công bố nguyên tử
4. Hành vi và vòng đời	BR-07 Phễu; BR-08 Customer 360; BR-09 Cohort, sản phẩm, địa lý	CRM, Analyst và Product	Hợp nhất định danh, lịch sử nhiều tháng, mô hình Gold
5–6. Quyền riêng tư, tính lại lịch sử	BR-10 Quản trị dữ liệu	Privacy và Data Engineer	Lọc consent trong đường ống, RBAC, snapshot Bronze bất biến

Chương 2

Dữ liệu và kiến trúc giải pháp

2.1 Nguồn dữ liệu

Dữ liệu marketing thật luôn gắn với thông tin cá nhân và bí mật kinh doanh, nên nhóm không dùng dữ liệu của doanh nghiệp có thật. Thay vào đó, gói `marketpulse_mock` sinh ra một "thế giới bán lẻ" mô phỏng có tính *tất định*: cùng một `seed` và cùng một cấu hình luôn cho ra cùng một bộ file, đến từng byte. Mỗi lần sinh dữ liệu đi kèm một `manifest.json` ghi lại cấu hình, mã băm SHA-256 của từng file, số dòng, mốc thời gian và số lượng bất thường đã tiêm vào. Nhờ vậy, mọi con số trong báo cáo đều có thể tái lập và đối chiếu lại với "sự thật gốc" (*ground truth*) của bộ sinh.

Định danh khách hàng, chiến dịch, phiên, click, đơn hàng và email được giữ nhất quán giữa các nguồn, nên dữ liệu vừa đủ "bẩn" để kiểm tra đường ống, vừa đủ chặt chẽ để đối soát. Bảng 2.1 tóm tắt các nguồn dữ liệu.

Bảng 2.1: *Đặc tả các nguồn dữ liệu, định dạng, khóa liên kết và đặc trưng nổi bật*

Nguồn	Định dạng / kênh	Khóa liên kết	Đặc trưng
`web_events`	JSONL, Kafka thời gian thực	`event_id`, `session_id`, `customer_id` hoặc `anonymous_id`, `click_id`, `campaign_id`	5 loại sự kiện phễu từ web và ứng dụng; có trùng lặp, đến trễ, phiên bot
`orders`	JSONL, Kafka thời gian thực	`order_id`, `customer_id`	`order_created`, `order_refunded`; số tiền là số nguyên VND
`email_events`	JSONL, Kafka thời gian thực	`message_id`, `customer_id`, mã chiến dịch vòng đời	Gửi, nhận, mở, click, hủy đăng ký
`google_ads`	CSV theo ngày	mã chiến dịch của nền tảng	Chi phí tính bằng `cost_micros`
`meta_ads`	JSONL theo ngày	mã chiến dịch của nền tảng	Chi phí là chuỗi thập phân, có cặp `date_start`/`date_stop`
`tiktok_ads`	CSV theo ngày	mã chiến dịch của nền tảng	Ngày ghi ở trường `stat_time_day`
`shopee_ads`	CSV theo ngày	mã quảng cáo	Ngày dạng `DD/MM/YYYY`, chi phí ở trường `expense`
`crm`	CSV, ảnh chụp hằng ngày	`customer_id`	Hạng thành viên, giá trị vòng đời, cờ đồng thuận
`master/*`	CSV	`campaign_id`, `ad_id`, `customer_id`, `product_id`	Danh mục chiến dịch, quảng cáo, khách hàng, sản phẩm

Luồng sự kiện thời gian thực

Ba luồng sự kiện được phát lại vào Kafka theo đúng thứ tự thời gian bởi công cụ `replay`. Luồng `web_events` mô tả hành trình mua sắm trên web và ứng dụng qua năm sự kiện `page_view`, `view_item`, `add_to_cart`, `begin_checkout` và `purchase`; các phiên đến từ quảng cáo trả phí mang theo `click_id` và `campaign_id`. Luồng `orders` ghi nhận giao dịch tài chính qua `order_created` và `order_refunded`. Luồng `email_events` theo dõi vòng đời một email qua `email_sent`, `email_delivered`, `email_opened`, `email_clicked` và `email_unsubscribed`.

Công cụ phát lại có hai đặc điểm quan trọng cho tính đúng. Thứ nhất, producer bật `enable.idempotence` và `acks=all`, nên việc gửi lại sau lỗi mạng không sinh bản ghi trùng trong Kafka. Thứ hai, con trỏ phát lại được gắn với mã băm của manifest: nếu thế giới dữ liệu thay đổi, con trỏ cũ sẽ bị từ chối thay vì âm thầm tiếp tục trên một tập dữ liệu khác.

Dữ liệu lô theo lịch

Nhóm dữ liệu lô mô phỏng đúng cách các nền tảng quảng cáo xuất báo cáo trong thực tế: mỗi nền tảng một định dạng, một cách ghi ngày và một đơn vị tiền riêng (Bảng 2.1). Bên cạnh đó là ảnh chụp CRM hằng ngày và bốn bảng danh mục `master/campaigns`, `master/ads`, `master/customers`, `master/products`. Tầng Silver chịu trách nhiệm chuẩn hóa toàn bộ các khác biệt này về một mô hình chung, quy chi phí về số nguyên VND bằng làm tròn *half-up*, và từ chối cả lô nếu có báo cáo chi phí không ánh xạ được về chiến dịch nào.

Các quy mô dữ liệu

Bộ sinh hỗ trợ nhiều cấu hình quy mô, phục vụ những mục đích khác nhau:

- **fixture**: 615 dòng trong 22 file nén, được lưu ngay trong kho mã để kiểm thử tự động và tái lập nhanh.
- **demo**: 1.200.225 dòng trong 130 file (khoảng 90 MB) với `seed` 360, dùng để kiểm tra đường ống ở quy mô vừa.
- **story-demo**: kịch bản `vietnam-retail` kéo dài 638 ngày, từ 01/01/2025 đến 30/09/2026, gồm 2.576.737 dòng trong 6.342 file. Kịch bản mô phỏng 13 tỉnh/thành phố, bốn kênh quảng cáo (kể cả Shopee Ads), nhịp truy cập theo giờ Việt Nam (GMT+7), các đợt cao điểm như 11/11, Black Friday và Tết, cùng lưu lượng tự nhiên, trực tiếp và từ email. Đây là bộ dữ liệu dùng cho phần minh chứng ứng dụng ở Chương 4.

2.2 Đặc trưng 4V

Khối lượng (Volume)

Ở quy mô story-demo, hệ thống lưu trữ và xử lý hơn 2,5 triệu bản ghi nguồn, trong đó riêng lịch sử sự kiện đến 31/08/2026 đã có 1.202.050 sự kiện, và lớp hành trình khách hàng sau quản trị còn 1.164.717 sự kiện hợp lệ. Khối lượng này đủ lớn để các lựa chọn về phân vùng bảng, định dạng cột và tách biệt tính toán–lưu trữ tạo ra khác biệt rõ rệt về chi phí xử lý; phần đánh giá ở Chương 5 còn mở rộng tầng phục vụ lên 20 triệu dòng.

Tốc độ (Velocity)

Sự kiện hành vi và đơn hàng đến liên tục, không theo lô. Khi phát lại lịch sử ở chế độ không giãn cách, toàn bộ 1.202.050 sự kiện được đẩy vào Kafka trong khoảng 32 giây, tức hơn 37.000 sự kiện mỗi giây, và Flink theo kịp với độ trễ tiêu thụ (consumer lag) bằng 0.

Ở chế độ trình diễn liên tục, tháng 09/2026 được phát lại với tốc độ gấp 20 lần thời gian thực để quan sát số liệu trực tiếp.

Đa dạng (Variety)

Dữ liệu đến từ chín loại nguồn với hai định dạng (CSV và JSONL), hai kiểu thời gian (luồng và ảnh chụp theo ngày), ba cách ghi ngày, ba cách biểu diễn tiền tệ và nhiều loại định danh (khách hàng đã đăng nhập, khách ẩn danh, phiên, click, đơn hàng, email). Thách thức lớn nhất của sự đa dạng là *hợp nhất định danh*: một khách hàng có thể bắt đầu ẩn danh trên ứng dụng, click quảng cáo trên web rồi đăng nhập khi thanh toán.

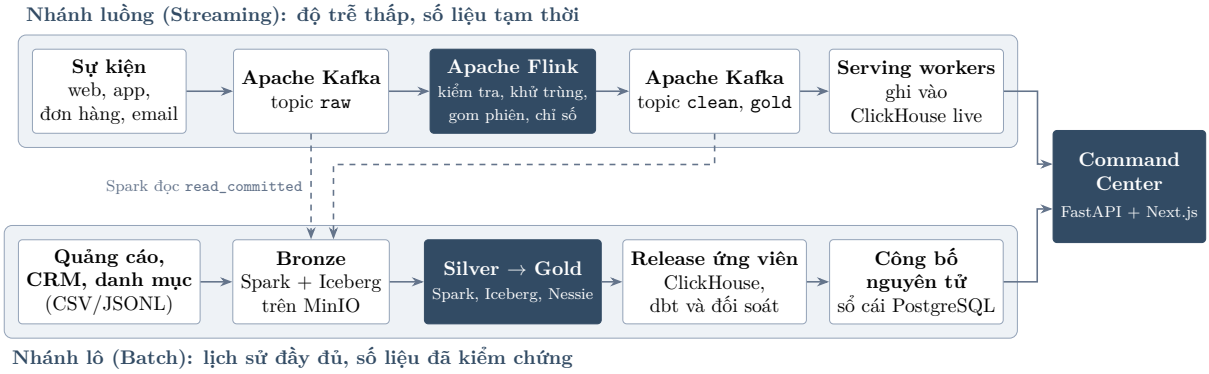
Độ tin cậy (Veracity)

Để kiểm chứng khả năng tự vệ của đường ống, bộ sinh tiêm nhiễu theo các tỷ lệ mặc định trong `DEFAULT_ANOMALY_RATES`: 0,5% sự kiện trùng lặp, 1,0% sự kiện đến trễ, 0,2% bản ghi sai quy cách, 0,2% phiên bot, 5,0% khách hàng từ chối đồng thuận phân tích và 5,0% đơn hàng bị hoàn tiền. Mỗi loại nhiễu có một cơ chế xử lý riêng ở những tầng khác nhau của đường ống, và số lượng thực tế của từng loại được ghi trong manifest để đối soát sau xử lý.

2.3 Kiến trúc tổng thể

Vòng đời dữ liệu bốn giai đoạn

Dữ liệu trong MarketPulse 360 đi qua bốn giai đoạn: *thu thập* (Kafka và kho lưu trữ đối tượng MinIO), *xử lý* (Flink cho luồng, Spark cho lô), *lưu trữ* (Iceberg cho lịch sử, ClickHouse cho phục vụ phân tích, PostgreSQL cho dữ liệu vận hành) và *phân phối* (FastAPI và giao diện Next.js). Điểm then chốt của kiến trúc là cả hai nhánh xử lý đều bắt nguồn từ *cùng một trục sự kiện Kafka* (Hình 2.1): số liệu trực tiếp và số liệu lịch sử vì vậy luôn được tính từ cùng một nguồn sự thật.



Hình 2.1: Kiến trúc tổng thể kết hợp Streaming và Batch

Nhánh luồng (Streaming)

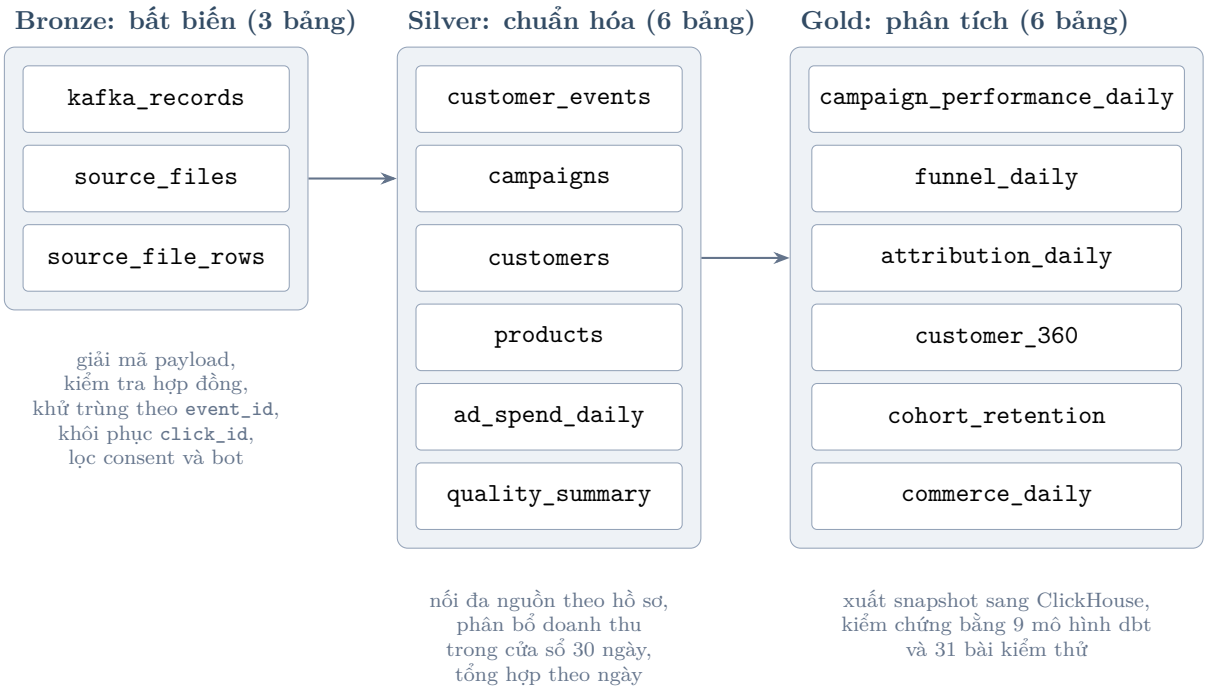
Flink đọc các topic `raw`, kiểm tra hợp đồng dữ liệu, khử trùng lặp, phân loại sự kiện trễ, gom phiên và cập nhật chỉ số theo chiến dịch, rồi ghi kết quả ra các topic `clean` và `gold` với bảo đảm *exactly-once*. Hai tiến trình nền của ứng dụng đọc các topic này và nạp vào cơ sở dữ liệu `marketpulse_live` trên ClickHouse. Số liệu ở nhánh này có độ trễ vài giây nhưng mang tính tạm thời: nó chưa qua lọc đồng thuận ở cấp hồ sơ, chưa có chi phí quảng cáo và chưa được đối soát.

Nhánh lô (Batch)

Theo lịch hằng ngày, Dagster điều phối Spark lưu trữ bất biến các bản ghi Kafka và các file báo cáo vào tầng Bronze, xây dựng tầng Silver và Gold trên Iceberg, xuất snapshot sang một cơ sở dữ liệu ClickHouse ứng viên, chạy `dbt build` cùng bộ đối soát, rồi mới công bố. Nhánh này chậm hơn nhưng đầy đủ: nó thấy dữ liệu đến trễ, có chi phí quảng cáo từ cả bốn nền tảng và áp dụng toàn bộ quy tắc quản trị. Giao diện luôn ghi rõ con số nào thuộc nhánh nào.

2.4 Mô hình dữ liệu Medallion

Hồ dữ liệu được tổ chức theo kiến trúc Medallion gồm 15 bảng Iceberg (Hình 2.2). Mỗi tầng có một trách nhiệm rõ ràng: Bronze giữ nguyên bản gốc để có thể phát lại và tính lại; Silver làm sạch, chuẩn hóa và quản trị; Gold tổng hợp sẵn cho các câu hỏi nghiệp vụ. Toàn bộ lược đồ được khai báo một lần trong `contracts.py` và dùng chung cho cả mã xây dựng bảng lẫn mã kiểm chứng.

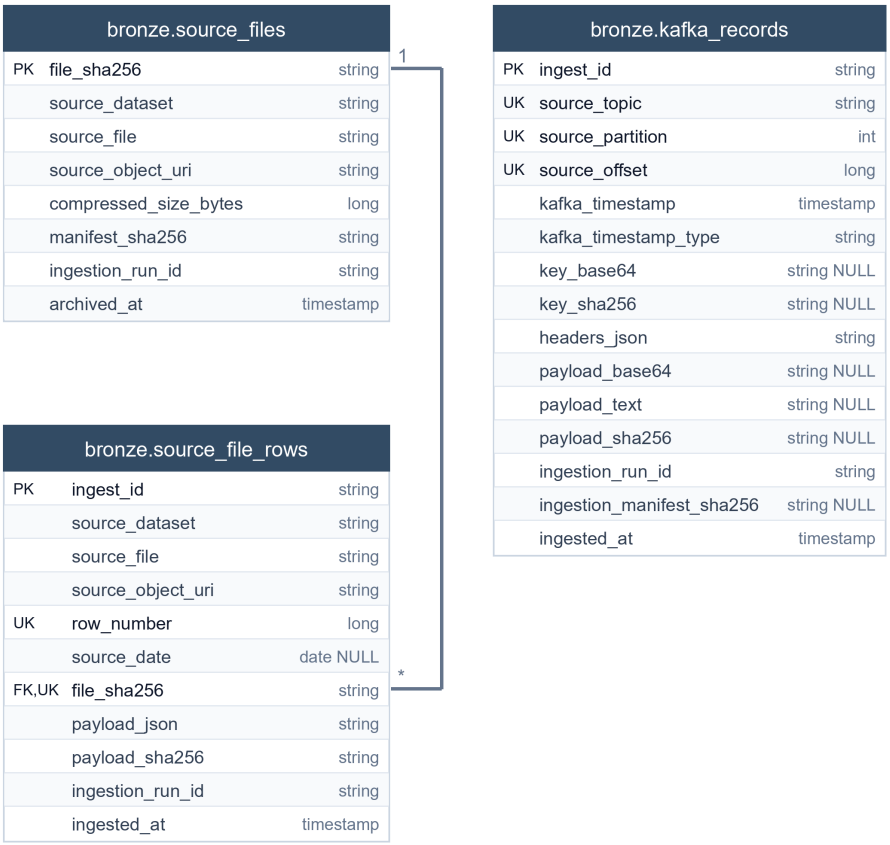


Hình 2.2: Luồng biến đổi dữ liệu Medallion Bronze → Silver → Gold

Trong các lược đồ quan hệ của báo cáo, khóa được ký hiệu `PK` (khóa chính), `FK` (khóa ngoại; chỉ số `FK1`, `FK2`… phân biệt các khóa ngoại trong cùng bảng) và `UK` (khóa duy nhất/khóa tự nhiên). Kiểu kèm chữ `NULL` là cột cho phép rỗng; các cột còn lại đều `NOT NULL`. Đường nét liền biểu diễn khóa ngoại, nét đứt biểu diễn tham chiếu không có ràng buộc (qua trường JSON hoặc tra cứu ở tầng ứng dụng), nét chấm có mũi tên biểu diễn quan hệ dẫn xuất. Với Iceberg và ClickHouse, khóa là *logic*: động cơ lưu trữ không cưỡng chế, nhưng đường ống kiểm tra chúng ở mỗi lần xây dựng.

Tầng Bronze: dữ liệu gốc bất biến

Tầng Bronze gồm ba bảng (Hình 2.3). Bảng `bronze.kafka_records` lưu nguyên văn từng bản ghi Kafka (khóa, header và payload dạng base64 kèm mã băm) cùng tọa độ `(source_topic, source_partition, source_offset)`. Khóa `ingest_id` được tính bằng SHA-256 của chính bộ tọa độ này, nên việc nạp lại cùng một khoảng offset không bao giờ tạo bản ghi trùng. Với dữ liệu file, `bronze.source_files` ghi mỗi file nguồn một dòng, khóa là mã băm nội dung `file_sha256`; file gốc được lưu vào MinIO theo địa chỉ nội dung. Bảng `bronze.source_file_rows` lưu từng dòng của file dưới dạng JSON, liên kết về file cha qua `file_sha256` theo quan hệ một–nhiều. Các bảng được phân vùng theo ngày Kafka hoặc theo tập dữ liệu và ngày nguồn để truy vấn lịch sử chỉ quét đúng phần cần thiết.

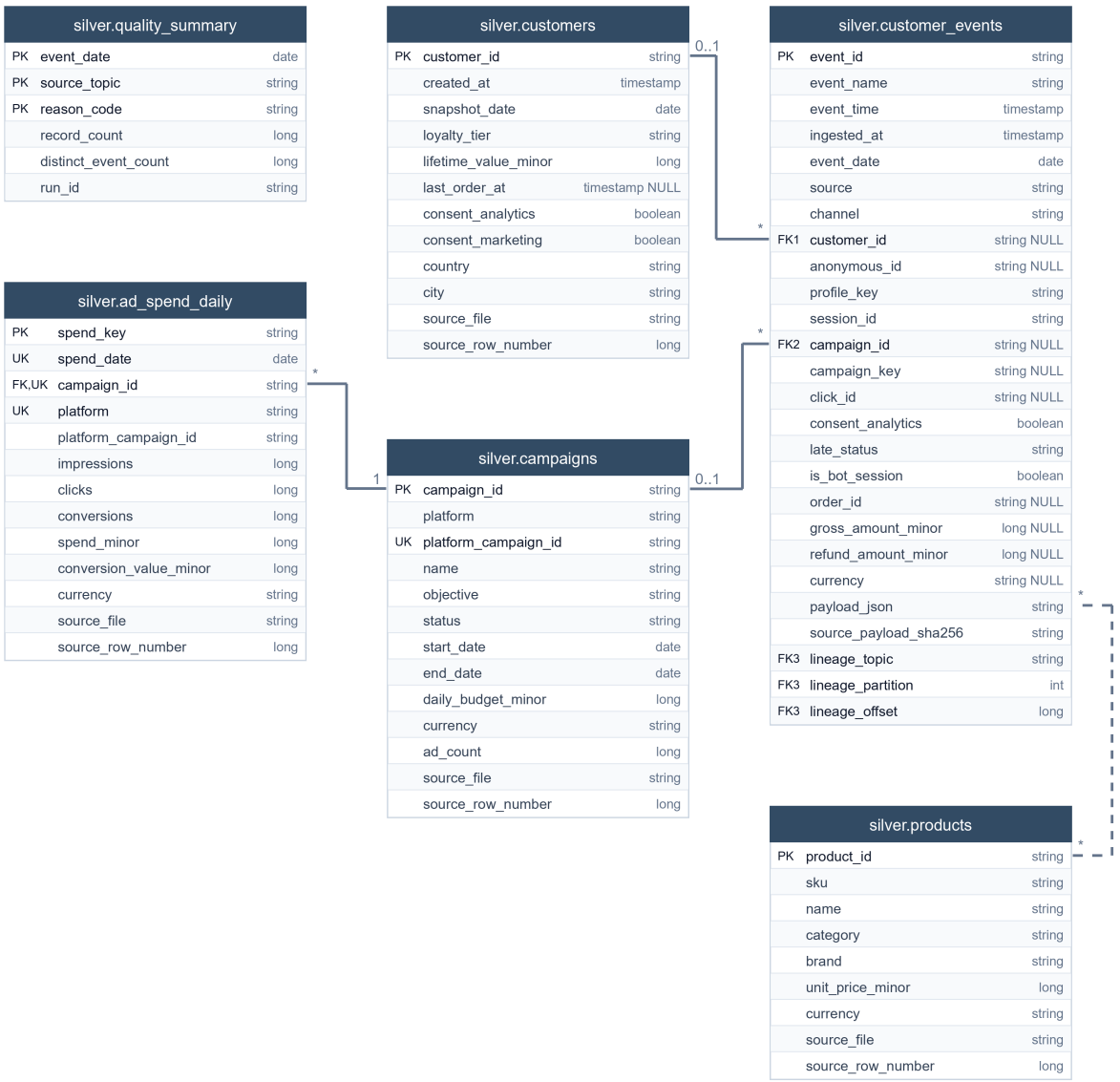


Hình 2.3: Lược đồ quan hệ tầng Bronze

Tầng Silver: dữ liệu chuẩn hóa

Tầng Silver gồm sáu bảng (Hình 2.4). Trung tâm là `silver.customer_events` với 26 cột: mỗi sự kiện hợp lệ xuất hiện đúng một lần theo `event_id`, mang `profile_key` đã hợp nhất (`customer:`, `anonymous:` hoặc `session:`), cờ bot, trạng thái đến trễ, và bộ ba lineage `(lineage_topic, lineage_partition, lineage_offset)` trỏ ngược về đúng bản ghi gốc trong Bronze. Ba bảng danh mục `silver.campaigns`, `silver.customers`, `silver.products` cùng bảng chi phí `silver.ad_spend_daily` đều giữ lineage `(source_file, source_row_number)` về dòng nguồn. Bảng `silver.quality_summary` đếm số bản ghi theo ngày, topic và mã lý do (chấp nhận, trùng lặp, sai quy cách, từ chối đồng thuận, bot…), là đầu vào cho màn hình sức khỏe dữ liệu.

Quan hệ giữa các bảng được kiểm tra ngay khi xây dựng: chi phí không ánh xạ được về chiến dịch làm thất bại cả lô, khách hàng trong CRM phải tồn tại trong danh mục khách hàng, và mã `platform_campaign_id` phải duy nhất.



Hình 2.4: Lược đồ quan hệ tầng Silver

2.5 Lựa chọn lưu trữ và phương án thay thế

Phân tách lưu trữ theo tải truy cập

MarketPulse 360 không dùng một cơ sở dữ liệu cho mọi việc. Mỗi kiểu tải truy cập được giao cho hệ thống phù hợp nhất (Bảng 2.2): kho đối tượng giá rẻ cho dữ liệu gốc, định dạng bảng có giao dịch cho lịch sử, CSDL cột cho truy vấn phân tích độ trễ thấp và CSDL quan hệ cho dữ liệu vận hành cần ràng buộc chặt. Nhờ tách biệt, việc tính lại toàn bộ lịch sử trên Spark không ảnh hưởng đến người dùng đang đọc ClickHouse, và dữ liệu nghiệp vụ của ứng dụng (tài khoản, cấu hình, phân đoạn) không lẫn vào kho phân tích.

Bảng 2.2: *Vai trò của các công cụ xử lý và các thành phần lưu trữ*

Thành phần	Loại	Vai trò trong MarketPulse 360
Apache Kafka 4.3.1	Xử lý (truyền tải)	Trục sự kiện chung, lưu nhật ký có thứ tự để phát lại
Apache Flink 2.2.1	Xử lý luồng	Kiểm tra, khử trùng, gom phiên, chỉ số trực tiếp, exactly-once
Apache Spark 4.0.4	Xử lý lô	Xây dựng Bronze, Silver, Gold trên Iceberg
Dagster 1.13	Điều phối	Lập lịch và quản lý phụ thuộc giữa 13 tài sản dữ liệu
dbt Core 1.12	Biến đổi, kiểm thử	Xây dựng 9 mô hình phục vụ và chạy kiểm thử trước công bố
MinIO	Lưu trữ đối tượng	File nguồn gốc và file dữ liệu Iceberg
Apache Iceberg 1.12.0	Định dạng bảng	Snapshot có giao dịch, phân vùng ẩn, truy vấn theo phiên bản
Project Nessie 0.109.0	Catalog	Quản lý danh mục bảng Iceberg, lưu trên PostgreSQL riêng
ClickHouse 26.8	CSDL cột (OLAP)	Phục vụ release đã kiểm chứng và số liệu trực tiếp
PostgreSQL 17.6	CSDL quan hệ (OLTP)	Tài khoản, cấu hình, cảnh báo, phân đoạn, sổ cái công bố

Tầng xử lý luồng: Flink và Kafka Streams

Kafka Streams là lựa chọn thay thế tự nhiên của Flink: nó là thư viện nhúng vào ứng dụng Java, không cần cụm riêng và cũng hỗ trợ exactly-once. Ưu điểm này đáng giá với các luồng nhỏ. Tuy nhiên, Flink cung cấp sẵn thời gian sự kiện với watermark, bộ hẹn giờ theo thời gian sự kiện để đóng phiên, trạng thái có TTL và cơ chế khôi phục từ checkpoint, đúng những gì đường ống luồng của MarketPulse 360 cần. Nhóm vì vậy chọn Flink làm động cơ chính và dùng Kafka Streams làm đối chứng trong một thực nghiệm có kiểm soát trên cùng khối lượng công việc (Mục 5.3).

Tầng phục vụ: ClickHouse, PostgreSQL, BigQuery

Với tầng phục vụ, phương án đối chứng là PostgreSQL, một CSDL lưu theo dòng, dùng chỉ mục B-tree. Nếu PostgreSQL đủ nhanh, hệ thống có thể bỏ bớt một thành phần. Thực nghiệm ở Mục 5.4 so sánh hai hệ thống trên cùng dữ liệu dạng Gold ở các quy mô 1, 10 và 20 triệu dòng. Google BigQuery cùng BigLake là phương án đám mây có thể đọc trực tiếp bảng Iceberg; phương án này được cân nhắc nhưng chưa được đo đạc vì giới hạn tài nguyên đám mây của nhóm, và báo cáo không đưa ra kết luận nào về nó.

Chương 3

Công nghệ quản lý và xử lý dữ liệu

Chương này trình bày bốn cụm công nghệ cốt lõi của MarketPulse 360. Mỗi cụm được mô tả theo cùng một trình tự: công nghệ là gì và đóng vai trò gì, nó vận hành ra sao trong hệ thống, và những điểm hiện thực đáng chú ý. Cuối chương là năm ví dụ minh họa truy vết trực tiếp trên dữ liệu và mã nguồn.

3.1 Apache Kafka và Apicurio Registry

Định nghĩa và vai trò

Apache Kafka là nền tảng truyền dữ liệu phân tán dựa trên *nhật ký chỉ ghi thêm* (append-only log) [2, 3]. Mỗi topic được chia thành nhiều partition; bản ghi trong một partition có thứ tự và được định vị bằng offset, nên người tiêu thụ có thể đọc lại từ bất kỳ vị trí nào. Trong MarketPulse 360, Kafka 4.3.1 chạy ở chế độ KRaft (không cần ZooKeeper) và đóng vai trò *trục sự kiện chung*: mọi sự kiện đi qua Kafka đúng một lần, rồi được cả nhánh luồng lẫn nhánh lô tiêu thụ độc lập. Apicurio Registry 3.3.3 [4] lưu các hợp đồng dữ liệu (JSON Schema) có phiên bản cho từng topic.

Cơ chế hoạt động

Hệ thống dùng mười topic có phiên bản `v1`, chia thành bốn nhóm theo mức độ xử lý (Bảng 3.1). Topic `raw` nhận dữ liệu nguồn từ công cụ phát lại; topic `clean` và `gold` do Flink ghi ra; hai topic `dlq` chứa bản ghi lỗi ở phía producer và phía Flink. Khóa phân vùng được chọn theo khóa nghiệp vụ (ưu tiên `customer_id`, sau đó là `session_id`, `order_id` hoặc `message_id`) để mọi sự kiện của cùng một khách hàng rơi vào cùng partition và giữ nguyên thứ tự.

Bảng 3.1: *Cấu hình các Kafka topic*

Topic	Partition	Chính sách lưu	Khóa / bên ghi
`marketing.raw.web-events.v1`	6	delete, 7 ngày	khách hàng hoặc phiên; replay
`marketing.raw.order-events.v1`	6	delete, 7 ngày	khách hàng hoặc đơn hàng; replay
`marketing.raw.email-events.v1`	6	delete, 7 ngày	khách hàng hoặc email; replay
`marketing.dlq.validation.v1`	6	delete, 7 ngày	ca lỗi; replay
`marketing.clean.customer-events.v1`	6	delete, 7 ngày	`identity_key`; Flink
`marketing.clean.sessions.v1`	6	compact, delete, 7 ngày	phiên; Flink
`marketing.gold.funnel-metrics.v1`	3	compact, delete, 30 ngày	`metric_key`; Flink
`marketing.gold.revenue-metrics.v1`	3	compact, delete, 30 ngày	`metric_key`; Flink
`marketing.gold.campaign-metrics.v1`	3	compact, delete, 30 ngày	`metric_key`; Flink
`marketing.dlq.flink-validation.v1`	6	delete, 7 ngày	mã lỗi; Flink

Topic phiên và topic `gold` dùng chính sách *compact*: Kafka chỉ giữ giá trị mới nhất của mỗi khóa, phù hợp với dữ liệu dạng "trạng thái hiện tại" như chỉ số của một chiến dịch trong ngày. Trước khi phát lại, mười hợp đồng JSON Schema được đăng ký lên Apicurio: hợp đồng chưa có thì tạo mới, hợp đồng đã có chỉ được thêm phiên bản khi nội dung thật sự thay đổi.

Điểm nổi bật trong MarketPulse 360

- **Gửi không trùng.** Producer bật `enable.idempotence` và `acks=all`; nếu lần gửi bị lỗi, việc gửi lại không tạo bản ghi trùng trong partition.
- **Phát lại an toàn.** Con trỏ phát lại lưu vị trí đã gửi và gắn với mã băm của manifest dữ liệu. Nếu xác nhận gửi không rõ ràng hoặc thế giới dữ liệu đã thay đổi, công cụ dừng lại thay vì đoán.
- **Tách biệt hợp đồng và dữ liệu.** Hợp đồng JSON Schema được quản lý tập trung, giúp bản ghi lỗi bị chặn ngay ở cửa vào và được chuyển sang DLQ thay vì làm hỏng các bảng phía sau.

3.2 Apache Flink

Định nghĩa và vai trò

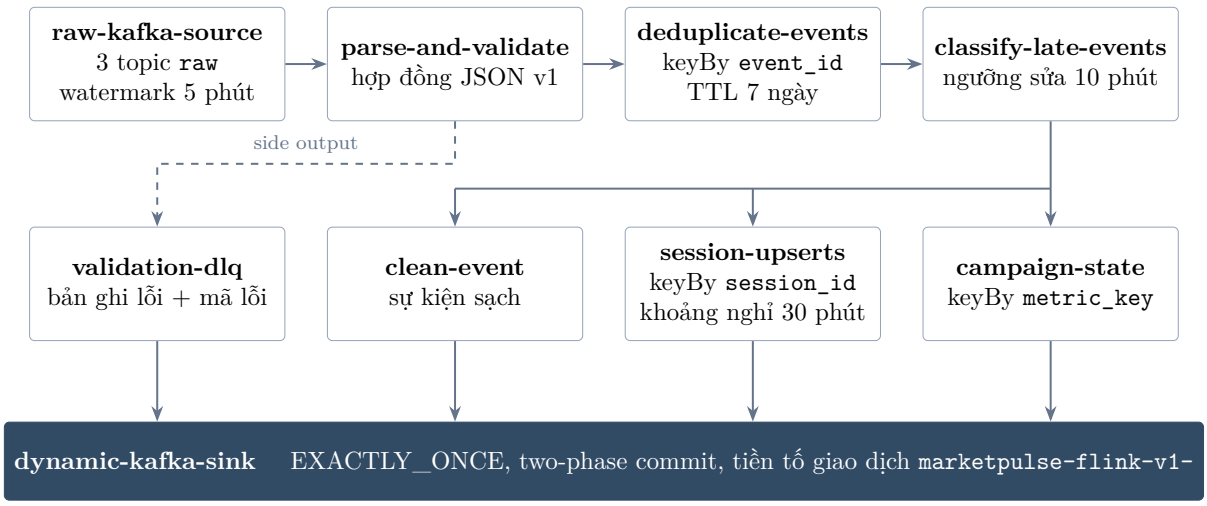
Apache Flink là động cơ xử lý luồng phân tán có trạng thái [5, 6]. Flink xử lý từng sự kiện ngay khi đến, cho phép lưu trạng thái theo khóa và khôi phục trạng thái đó sau sự

cố nhờ cơ chế checkpoint nhất quán [7]. Flink hỗ trợ *thời gian sự kiện* với watermark (ý tưởng xuất phát từ mô hình Dataflow [8]) nên xử lý đúng cả khi sự kiện đến không theo thứ tự. Trong MarketPulse 360, Flink 2.2.1 là bộ não của nhánh luồng: làm sạch sự kiện và tính chỉ số chiến dịch trong vài giây.

Cơ chế hoạt động

Job `marketpulse-stream-v1` gồm một chuỗi toán tử có định danh ổn định (Hình 3.1):

1. **Đọc và gán thời gian sự kiện.** Nguồn Kafka đọc ba topic `raw`; watermark cho phép sự kiện lệch tối đa 5 phút và đánh dấu partition nhàn rỗi sau 1 phút để watermark không bị kẹt.
2. **Kiểm tra hợp đồng.** Bản ghi không đúng hợp đồng được tách sang side output và ghi vào `marketing.dlq.flink-validation.v1` kèm mã lỗi, không làm dừng job.
3. **Khử trùng lặp.** Luồng được phân theo `event_id`; trạng thái `ValueState` ghi nhớ sự kiện đã thấy trong 7 ngày (TTL 604.800.000 ms) và loại mọi bản sao.
4. **Phân loại sự kiện trễ.** Sự kiện đến sau watermark nhưng trong ngưỡng sửa 10 phút vẫn được cập nhật vào chỉ số; sự kiện trễ hơn được ghi nhận để kiểm toán.
5. **Gom phiên.** Theo `session_id`, bộ hẹn giờ theo thời gian sự kiện đóng phiên sau 30 phút không hoạt động.
6. **Chỉ số chiến dịch.** Theo `metric_key` (chiến dịch × ngày), trạng thái tích lũy số phiên ở từng bước phễu, số đơn, doanh thu gộp, hoàn tiền và doanh thu ròng.
7. **Ghi exactly-once.** Một sink Kafka duy nhất định tuyến bản ghi tới đúng topic và ghi theo giao thức two-phase commit với tiền tố giao dịch `marketpulse-flink-v1-`. Checkpoint chạy mỗi 5 giây ở chế độ exactly-once.



Hình 3.1: Topology chuỗi toán tử xử lý luồng có trạng thái trên Apache Flink

Điểm nổi bật trong MarketPulse 360

Điểm quan trọng nhất là *tính nhất quán khi có sự cố*. Offset nguồn, trạng thái khử trùng, trạng thái phiên và chỉ số chiến dịch được lưu trong cùng một checkpoint, còn kết quả chỉ hiển thị cho người đọc sau khi giao dịch Kafka được commit. Vì vậy, khi một TaskManager bị khởi động lại, job tiếp tục từ checkpoint gần nhất mà không đếm thiếu hay đếm trùng (Ví dụ 1, Mục 3.5). Mọi toán tử đều có `uid` cố định, nên có thể nâng cấp job mà vẫn khôi phục được trạng thái cũ. Bộ gửi job cũng được thiết kế lũy đẳng: nó nhận ra job đang chạy ở mọi trạng thái chưa kết thúc và không tạo job thứ hai.

3.3 Lakehouse: MinIO, Iceberg, Nessie và Spark

Định nghĩa và vai trò

Kiến trúc *lakehouse* kết hợp chi phí thấp của hồ dữ liệu với tính giao dịch của kho dữ liệu [9]. MarketPulse 360 hiện thực kiến trúc này bằng bốn thành phần: MinIO là kho đối tượng tương thích S3 [10]; Apache Iceberg 1.12.0 là định dạng bảng mở với snapshot có giao dịch và phân vùng ẩn [11]; Project Nessie 0.109.0 là catalog quản lý danh mục bảng [12]; Apache Spark 4.0.4 là động cơ tính toán lô [13, 14].

Cơ chế hoạt động

Tầng Bronze. Spark đọc Kafka ở chế độ `read_committed` theo một khoảng offset xác định, nên chỉ thấy bản ghi của giao dịch đã commit. Việc ghi dùng `MERGE` theo khóa `ingest_id`; nếu cùng một khóa xuất hiện với nội dung khác, quá trình dừng với lỗi va chạm khóa bất biến. File nguồn được kiểm tra kích thước và mã băm so với manifest trước khi lưu.

Tầng Silver. Spark giải mã các sự kiện `clean`, gom các bản sao theo `event_id` và chọn bản thắng theo thứ tự xác định. Phiên bản 1 của hợp đồng `clean` không mang `click_id`, nên Silver nối ngược về bản ghi `raw` qua bộ ba lineage để khôi phục trường này, đồng thời kiểm tra `event_id` hai phía phải trùng nhau. Sự kiện không có đồng thuận phân tích bị loại với mã `CONSENT_DENIED`, phiên bot web/app bị loại với mã `BOT_EXCLUDED`; mọi quyết định đều được đếm trong `silver.quality_summary`.

Tầng Gold. Spark xây dựng sáu bảng phân tích. Phép phân bổ doanh thu tìm các click trả phí của cùng hồ sơ trong cửa sổ 30 ngày trước đơn hàng, rồi chia tiền theo ba mô hình First-Touch, Last-Touch và Linear; đơn hàng không có điểm chạm nào được gán cho `__unattributed__` để tổng tiền luôn bảo toàn.

Điểm nổi bật trong MarketPulse 360

- **Tái lập theo snapshot.** Mỗi lần chuẩn bị dữ liệu sinh một *phiếu chuẩn bị* (preparation receipt) ghi lại mã snapshot của từng bảng Bronze, Silver và Gold. Nhờ

truy vấn theo phiên bản của Iceberg (`VERSION AS OF`), mọi tính toán sau đó đều đọc đúng trạng thái đã ghim, kể cả khi bảng nguồn đã có dữ liệu mới.

- **Không gian tên cô lập.** Mỗi lần chuẩn bị ghi Silver và Gold vào một namespace riêng `mp_batch_<identity>_<attempt>`, nên một lần chạy lỗi không bao giờ làm hỏng kết quả trước đó.
- **Tiền là số nguyên.** Mọi số tiền được lưu bằng đơn vị nhỏ nhất (VND) kiểu `long`; phép chia luôn là phép chia nguyên có phân bổ phần dư (Ví dụ 3).
- **Phân vùng phù hợp truy vấn.** Bảng sự kiện phân vùng theo tháng, bảng Kafka phân vùng theo ngày, giúp truy vấn lịch sử và tính lại chỉ quét đúng phần dữ liệu liên quan.

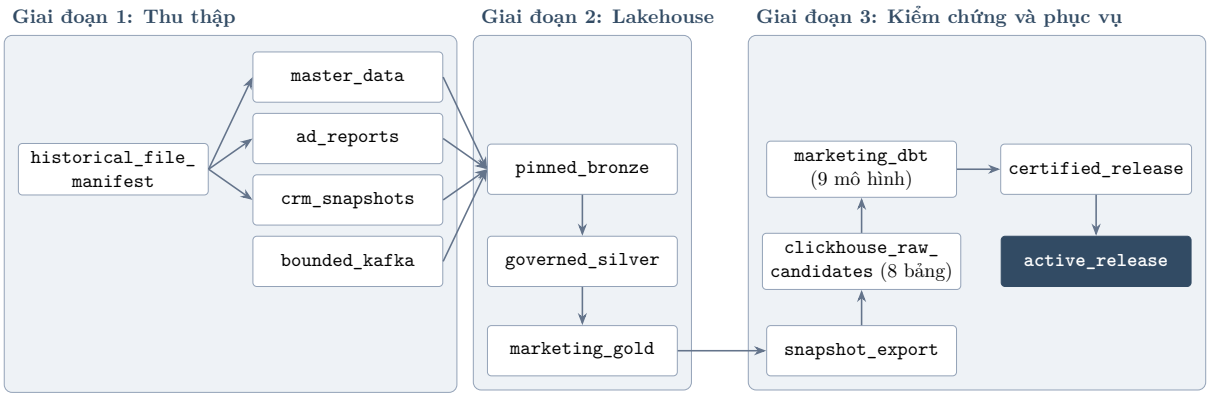
3.4 Dagster, dbt và ClickHouse

Định nghĩa và vai trò

Dagster 1.13 là công cụ điều phối theo mô hình *tài sản dữ liệu định nghĩa bằng phần mềm* (Software-Defined Assets): thay vì mô tả các tác vụ, người phát triển mô tả các bảng cần có và phụ thuộc giữa chúng [15]. dbt Core 1.12 biến đổi dữ liệu bằng SQL có quản lý phiên bản và chạy kiểm thử dữ liệu như kiểm thử phần mềm [16]. ClickHouse 26.8 là CSDL lưu theo cột tối ưu cho truy vấn phân tích, dùng họ động cơ bảng MergeTree [17].

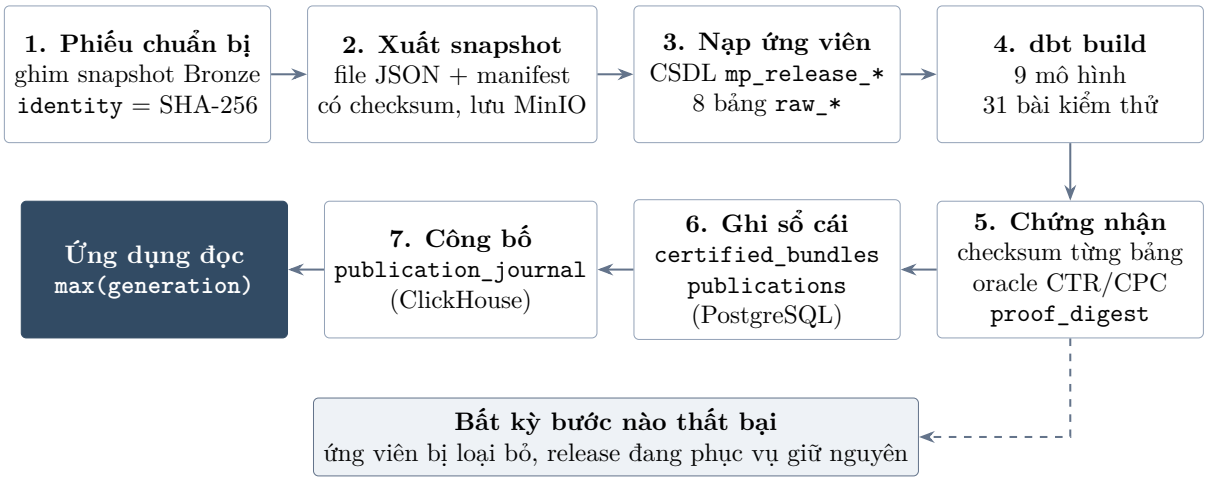
Cơ chế hoạt động

Đồ thị gồm 13 định nghĩa tài sản chia thành ba giai đoạn (Hình 3.2). Lịch `daily_marketing_refresh` chạy lúc 02:00 hằng ngày cho ngày hôm trước thông qua job `marketing_refresh_job`. Job `historical_rebuild_job` chạy lại toàn bộ đồ thị trừ bước công bố, dùng để tính lại lịch sử một cách cô lập; job `source_history_job` nạp lần đầu dữ liệu nguồn cho các ngày quá khứ.



Hình 3.2: Đồ thị phụ thuộc các tài sản dữ liệu trên Dagster

Sau khi Gold được xây xong, snapshot được xuất thành file JSON kèm manifest có checksum, rồi nạp vào một CSDL ClickHouse *ứng viên* mới tinh `mp_release_<identity>_<attempt>`. Tám bảng `raw_*` trong CSDL này là nguồn của dbt; dbt dựng chín mô hình và chạy 31 bài kiểm thử (Bảng 3.2). Chỉ khi mọi bước của quy trình trong Hình 3.3 thành công, bản phát hành mới được công bố.



Hình 3.3: Quy trình kiểm chứng bản phát hành cô lập và công bố nguyên tử

Tầng Gold và các mô hình dbt

Hình 3.4 trình bày chín mô hình trong CSDL release, với kiểu dữ liệu ClickHouse. Sáu mô hình là bản sao chính xác của các bảng Iceberg `gold.*`; `campaigns` và `customers` là hai bảng chiều xuất từ Silver; `campaign_dashboard` là mart làm giàu từ `campaign_performance_daily` (32 cột) và `campaigns`, bổ sung tên, trạng thái chiến dịch, CTR và CPC. Khóa ngoại tới `campaigns` được kiểm thử bởi `campaign_relationships`, ngoại trừ giá trị đặc biệt `__unattributed__`; khóa ngoại tới `customers` được kiểm thử bởi `customer_relationships`. Bảng `cohort_retention` được tổng hợp từ `customer_360` và hoạt động theo tháng nên được biểu diễn bằng quan hệ dẫn xuất.



Hình 3.4: Lược đồ quan hệ tầng Gold và 9 mô hình dbt phục vụ trên ClickHouse

Bảng 3.2: *Các mô hình dbt và bài kiểm thử chất lượng dữ liệu tự động*

Mô hình (ORDER BY)	**Nguồn**	**Kiểm thử tổng quát**
`campaign_performance_daily`	`raw_campaign_performance_daily`	khóa duy nhất, cột bắt buộc
`funnel_daily`	`raw_funnel_daily`	khóa duy nhất, cột bắt buộc, not null
`attribution_daily`	`raw_attribution_daily`	khóa duy nhất, cột bắt buộc, giá trị hợp lệ
`customer_360`	`raw_customer_360`	khóa duy nhất, cột bắt buộc, giá trị hợp lệ
`cohort_retention`	`raw_cohort_retention`	khóa duy nhất, cột bắt buộc
`commerce_daily`	`raw_commerce_daily`	khóa duy nhất, cột bắt buộc
`campaigns`	`raw_campaigns`	khóa duy nhất, cột bắt buộc, not null
`customers`	`raw_customers`	khóa duy nhất, cột bắt buộc, not null
`campaign_dashboard`	`campaign_performance_daily` $\bowtie$ `campaigns`	khóa duy nhất
Kiểm thử nghiệp vụ (9): `financial_equations` (gộp − hoàn = ròng ở mọi bảng), `attribution_conservation` (tổng phân bổ của mỗi mô hình bằng doanh thu Customer 360), `commerce_conservation`, `campaign_relationships`, `customer_relationships`, `cohort_invariants`, `funnel_invariants`, `consent_eligibility`, `accepted_domains`.		
Tổng cộng: 22 kiểm thử tổng quát + 9 kiểm thử nghiệp vụ = 31 kiểm thử.		

Điểm nổi bật trong MarketPulse 360

- **Kiểm chứng trên dữ liệu thật của release.** Ngoài 31 bài kiểm thử dbt, bước chứng nhận tính checksum cho toàn bộ dòng của từng bảng và so với checksum lúc xuất, tính lại CTR, CPC bằng một oracle độc lập, rồi gộp mọi bằng chứng thành `proof_digest`.
- **Công bố nguyên tử và đơn điệu.** Sổ cái PostgreSQL `marketpulse_ops` giữ khóa tư vấn (advisory lock) khi đặt chỗ công bố và từ chối bản phát hành của một ngày logic cũ hơn bản đã công bố. Bản ghi được nối vào `publication_journal` trên ClickHouse rồi mới được xác nhận; ứng dụng luôn đọc bản có `generation` lớn nhất. Một tiến trình công bố chậm vì thế không thể "kéo lùi" người đọc về bản cũ.
- **Không tái sử dụng ứng viên dở dang.** Mỗi lần nạp tạo một CSDL mới theo số lần thử; một CSDL nạp dở không bao giờ được dùng lại.

3.5 Ví dụ minh họa

Năm ví dụ dưới đây được chọn để mỗi ví dụ minh họa một rủi ro điển hình của dữ liệu marketing và cách đường ống xử lý nó. Các con số đều lấy từ bằng chứng chạy thật

trong kho mã [1].

Ví dụ 1: Khử trùng lặp và phục hồi trên Flink

Kịch bản kiểm thử phục hồi khởi động lại TaskManager của Flink *trong khi* producer vẫn đang gửi dữ liệu, kèm hai bản sao trùng `event_id` được chèn có chủ đích. Trong quá trình phục hồi, đầu vào Kafka thô tăng từ 83 lên 219 bản ghi. Sau khi job khôi phục từ checkpoint, đầu ra giữ đúng 217 mã sự kiện duy nhất và 73 khóa phiên, hai bản sao bị loại, và doanh thu ròng đối soát khớp 24.751.000 VND. Kết quả cho thấy trạng thái khử trùng được khôi phục cùng offset nguồn: bản ghi đã xử lý trước sự cố không bị tính lại, bản ghi đến trong lúc sự cố không bị mất.

Ví dụ 2: Khôi phục mã click tại tầng Silver

Một phiên mua hàng bắt đầu từ click quảng cáo trả phí có `click_id` trong bản ghi `raw`, nhưng bản ghi `clean` phiên bản 1 không mang trường này. Silver dùng bộ ba `(lineage_topic, lineage_partition, lineage_offset)` để tìm đúng bản ghi `raw` trong `bronze.kafka_records`, đọc payload, kiểm tra `event_id` trùng khớp rồi mới khôi phục `click_id`; nhờ đó đơn hàng được phân bổ về đúng chiến dịch.

Cùng lúc, Silver áp dụng quản trị. Trên tập fixture, doanh thu ròng của luồng chưa quản trị là 24.751.000 VND, còn doanh thu đã quản trị được công bố là 22.193.000 VND; hai con số thuộc hai quần thể khác nhau và không được so sánh như nhau. Trên story-demo, dữ liệu nguồn có 33.858 đơn hàng, bản phát hành đã kiểm chứng có 32.774 đơn; chênh lệch đúng bằng 1.084 đơn hàng có `consent_analytics = false` và bị loại với mã `CONSENT_DENIED`.

Ví dụ 3: Phân bổ tuyến tính bảo toàn từng đồng

Giả sử một đơn hàng có doanh thu gộp $A = 1.000.000$ VND và có $k = 3$ click trả phí trong cửa sổ 30 ngày. Mô hình Linear chia đều nhưng 1.000.000 không chia hết cho 3. Hàm `allocate_minor_units` sắp xếp các điểm chạm theo `(event_time, campaign_id, click_id, event_id)` rồi tính

$$q = \left\lfloor \frac{A}{k} \right\rfloor = 333.333, \qquad r = A \bmod k = 1.$$

r điểm chạm sớm nhất nhận thêm 1 đồng: phần phân bổ là 333.334, 333.333, 333.333 VND, tổng đúng bằng 1.000.000 VND. Hoàn tiền được phân bổ theo cùng cách, và doanh thu ròng của mỗi phần bằng gộp trừ hoàn. Bài kiểm thử `attribution_conservation` xác nhận tổng phân bổ của cả ba mô hình đều bằng tổng doanh thu trong `customer_360`.

Ví dụ 4: Chặn bản phát hành lỗi

Kiểm thử phương trình tài chính chỉ bắt được dòng có gộp − hoàn ≠ ròng. Một lỗi tinh vi hơn là dữ liệu bị sửa *nhất quán*: số dòng không đổi và phương trình vẫn cân. API `POST /api/v1/ops/quality/probe` (chỉ dành cho `admin`) tái hiện đúng tình huống này một cách an toàn: hệ thống sao chép release đang phục vụ sang một CSDL dùng một lần `mp09_probe_<uuid>`, cộng thêm 1 đồng vào cả doanh thu gộp lẫn doanh thu ròng của một dòng trong `campaign_performance_daily`, rồi chạy lại các phép kiểm chứng. Lỗi bị phát hiện ở hai lớp độc lập: checksum của bảng không còn khớp với checksum đã chứng nhận, và tổng doanh thu không còn khớp với tổng tính trực tiếp từ sự kiện đơn hàng. Báo cáo kết luận `controlled_same_count_finance_corruption_detected`, CSDL thử nghiệm bị xóa, còn release đang phục vụ không bị đụng tới. Trong quá trình kiểm chứng thật, các lỗi tương tự, như dòng tiền bị sửa với số dòng không đổi hoặc tham chiếu chiến dịch không tồn tại, đều làm bản ứng viên bị từ chối.

Ví dụ 5: Tái tạo lịch sử từ Iceberg snapshot

Khi cần kiểm tra hoặc tính lại một ngày trong quá khứ, `historical_rebuild_job` đọc phiếu chuẩn bị đã lưu của ngày đó. Job chỉ chạy nếu phiên bản mô hình hiện tại trùng với phiên bản đã ghi trong phiếu; nó cấp một lần thử mới và dựng lại Silver, Gold trong namespace cô lập `mp_batch_<identity>_<attempt>` từ đúng các snapshot Bronze đã ghim. Kết quả được so sánh với `proof_digest` đã ghi; nếu khác nhau, job thất bại. Bước công bố bị bỏ qua hoàn toàn trong chế độ này, nên việc tính lại lịch sử không bao giờ thay đổi số liệu người dùng đang xem.

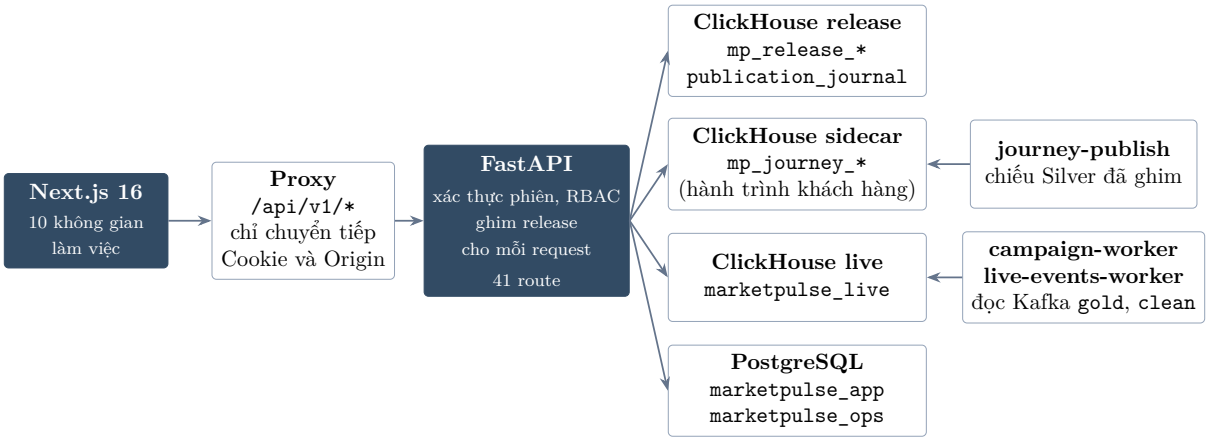
Chương 4

Ứng dụng Marketing Command Center

Dữ liệu chỉ tạo ra giá trị khi đến được tay người ra quyết định. Chương này trình bày cách MarketPulse 360 đưa kết quả của đường ống tới người dùng qua ứng dụng *Marketing Command Center*: kiến trúc phục vụ, thiết kế cơ sở dữ liệu vận hành, và kết quả khai thác thực tế cho từng yêu cầu nghiệp vụ trên bộ dữ liệu story-demo.

4.1 Kiến trúc ứng dụng

Ứng dụng gồm giao diện Next.js [18], một proxy nội bộ và tầng API FastAPI [19] (Hình 4.1). Trình duyệt không bao giờ gọi thẳng vào API: mọi lời gọi đi qua route `/api/v1/*` của Next.js, nơi chỉ chuyển tiếp cookie phiên, header `Origin` và kiểu nội dung. FastAPI chịu trách nhiệm xác thực, phân quyền và truy vấn ba nhóm kho dữ liệu tách biệt.



Hình 4.1: Kiến trúc tầng phục vụ ứng dụng Marketing Command Center

API FastAPI và ghim phiên bản đọc

Tầng API cung cấp 41 route, gồm các nhóm bảng điều khiển, phân tích hành trình, vòng đời khách hàng, insight, vận hành và xác thực. Mỗi request đọc số liệu đã kiểm chứng

đều *ghim* một bản phát hành: API tra `publication_journal` lấy bản có `generation` lớn nhất, rồi dùng đúng CSDL `mp_release_*` của bản đó cho mọi truy vấn trong request. Nhờ vậy, nếu một bản phát hành mới được công bố giữa chừng, người dùng vẫn nhận được kết quả nhất quán từ một bản duy nhất. Mọi phản hồi đều kèm thông tin nguồn gốc (mã release, ngày dữ liệu) để giao diện hiển thị độ tươi của số liệu. Số tiền được truyền dưới dạng chuỗi số nguyên để tránh sai số dấu phẩy động.

Tiến trình nền cho dữ liệu trực tiếp

Ba tiến trình nền bổ sung dữ liệu cho ứng dụng. `campaign-worker` đọc topic `marketing.gold.campaign-metrics.v1` và nạp vào `marketpulse_live.campaign_snapshots`; `live-events-worker` đọc `marketing.clean.customer-events.v1` và nạp vào `marketpulse_live.events`. Cả hai dùng chế độ `read_committed` và commit offset thủ công sau khi ghi thành công, nên một lần gửi lại chỉ trùng về vị trí và được loại theo khóa `(partition, offset)`. Tiến trình `journey-publish` chạy một lần cho mỗi release: nó đọc `silver.customer_events` tại đúng snapshot đã ghim, chỉ giữ sự kiện có đồng thuận và không thuộc phiên bot, thay `profile_key` bằng bút danh `profile_ref` rồi ghi vào CSDL sidecar `mp_journey_*`.

Xác thực, phân quyền và kiểm toán

Ứng dụng không có tài khoản mặc định và không có công tắc tắt xác thực. Tài khoản do quản trị viên tạo bằng dòng lệnh; mật khẩu chỉ được lưu dưới dạng băm scrypt có salt ngẫu nhiên. Khi đăng nhập, máy chủ cấp một token ngẫu nhiên 32 byte trong cookie `HttpOnly`, `SameSite=Strict`; cơ sở dữ liệu chỉ lưu SHA-256 của token, với thời hạn tuyệt đối 8 giờ. Đăng nhập sai quá 5 lần trong 15 phút sẽ bị chặn tạm thời. Ba vai trò được phân cấp rõ ràng:

- `viewer`: xem các bảng điều khiển tổng hợp, phễu, phân bổ, cohort, cảnh báo và mô phỏng chỉ đọc;
- `operator`: thêm quyền xem hồ sơ và dòng thời gian khách hàng, tạo phân đoạn, sửa ngân sách và quy tắc cảnh báo;
- `admin`: thêm quyền xem sức khỏe hệ thống, chạy kiểm chứng chất lượng và quản lý tài khoản.

Kiểm tra quyền diễn ra ở máy chủ *trước* mọi truy vấn dữ liệu. Mọi quyết định truy cập được ghi vào bảng `access_audit` chỉ cho phép thêm mới; một trigger sẽ từ chối mọi lệnh `UPDATE` và `DELETE`.

4.2 Cơ sở dữ liệu vận hành

Dữ liệu do người dùng ứng dụng tạo ra (tài khoản, cấu hình, quy tắc, phân đoạn) có bản chất giao dịch: cần ràng buộc chặt, cập nhật có kiểm soát phiên bản và nhật ký bất biến. Phần này vì vậy được đặt trên PostgreSQL 17.6 [20], tách hẳn khỏi kho phân tích.

Lược đồ marketpulse_app (PostgreSQL)

Lược đồ `marketpulse_app` có 12 bảng, chia thành bốn nhóm (Hình 4.2):

- **Bảo mật:** `auth_users`, `auth_sessions` (khóa ngoại tới người dùng), `login_failures` (bộ đếm theo cửa sổ 15 phút) và `access_audit` (nhật ký bất biến).
- **Phân đoạn khách hàng:** `segments` giữ phiên bản hiện hành, mỗi lần sửa sinh một dòng mới trong `segment_revisions` (khóa chính kép `(segment_id, version)`, bất biến nhờ trigger). Phân đoạn và phiên bản đầu tiên được ghi trong cùng một giao dịch, nên mỗi phân đoạn luôn có ít nhất một phiên bản.
- **Ngân sách và cảnh báo:** `campaign_settings`, `alert_rules`, `alert_evaluations` và `alert_occurrences`. Khóa `evaluation_key` là SHA-256 của quy tắc, phiên bản, cửa sổ thời gian và release, nên một lần đánh giá chạy lại không tạo cảnh báo trùng.
- **Vận hành:** `quality_reports` lưu kết quả kiểm chứng (bất biến) và `ops_state` là bảng một dòng.

Đáng chú ý, `campaign_id` trong cấu hình và quy tắc cảnh báo *không* là khóa ngoại: chiến dịch nằm trong kho phân tích ClickHouse, nên API kiểm tra chiến dịch có tồn tại trong release đang phục vụ trước khi ghi. Các thao tác ghi dùng khóa lạc quan theo cột `version` để hai người cùng sửa không ghi đè lẫn nhau.

Lược đồ marketpulse_ops (PostgreSQL)

Lược đồ `marketpulse_ops` nằm trên một instance PostgreSQL riêng của khối batch và gồm 4 bảng sổ cái của quy trình công bố. `preparations` lưu phiếu chuẩn bị bất biến; `publications` và `certified_bundles` có quan hệ 1–0..1 với `preparations` qua khóa ngoại thật; `attempts` đếm số lần thử theo giai đoạn và được tham chiếu mềm, vì lần thử đầu tiên được cấp *trước* khi phiếu chuẩn bị tồn tại.



Hình 4.2: Lược đồ quan hệ PostgreSQL: marketpulse_app và marketpulse_ops

Các bảng phục vụ trên ClickHouse

Ngoài các CSDL release, ClickHouse còn chứa ba nhóm bảng phục vụ ứng dụng (Hình 4.3). `marketpulse_serving.publication_journal` là nhật ký chỉ ghi thêm, phản chiếu bảng `publications` của sổ cái; `journey_publications` ghi nhận CSDL hành trình của từng release. Trong CSDL sidecar, bảng `events` lưu hành trình đã ẩn danh và view `paid_touches` chọn click trả phí đầu tiên cho mỗi cặp `(profile_ref, click_id)`. Kho `marketpulse_live` gồm `campaign_snapshots` (MergeTree), `events` (ReplacingMergeTree theo `(partition, offset)`) và hai bảng một dòng ghim định danh nguồn Kafka: nếu cụm Kafka hoặc topic bị thay đổi, worker từ chối ghi thay vì trộn hai nguồn.



Hình 4.3: Lược đồ quan hệ các bảng ClickHouse phục vụ, journey sidecar và kho dữ liệu trực tiếp

4.3 Giao diện và yêu cầu nghiệp vụ

Kết quả trong phần này được đo trên release `109a2372…` của bộ dữ liệu story-demo, gồm lịch sử đã kiểm chứng từ 01/01/2025 đến 31/08/2026 và luồng trực tiếp của tháng 09/2026. Mọi con số tổng hợp đều đã được một oracle độc lập tính lại từ file nguồn và khớp hoàn toàn (Mục 5.2).

Tổng quan giao diện

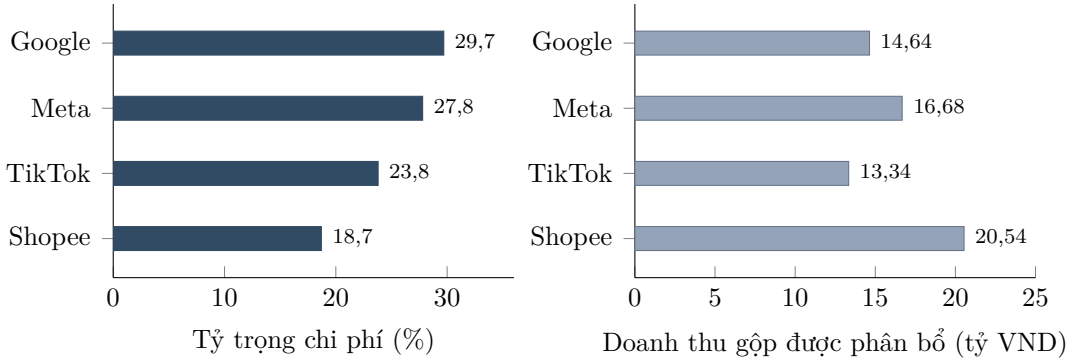
Giao diện được tổ chức thành 10 không gian làm việc, mỗi không gian phục vụ một nhóm câu hỏi (Bảng 4.1). Nguyên tắc thiết kế xuyên suốt là *trung thực về dữ liệu*: số liệu trực tiếp được gắn nhãn tạm thời và không bao giờ cộng với số liệu đã kiểm chứng; khi thiếu dữ liệu, giao diện hiển thị "không có dữ liệu" thay vì số 0; ngày tháng ghi rõ múi giờ.

Bảng 4.1: *Các không gian làm việc của Marketing Command Center*

Không gian	Đường dẫn	Nội dung chính
Overview	`/`	KPI chiến dịch đã kiểm chứng, số liệu trực tiếp, cảnh báo, giải ngân, mô phỏng
Live View	`/live`	Khách truy cập, phiên, đơn hàng của ngày hiện tại từ luồng trực tiếp
Ad Performance	`/ads`	Chi phí, click, ROAS theo nền tảng và chiến dịch
Analysis	`/analysis`	Phễu chuyển đổi, so sánh mô hình phân bổ, hành trình mẫu
Customers	`/customers`	Customer 360, dòng thời gian, phân đoạn, cohort, cơ hội và rủi ro
Email	`/email`	Lượng gửi, người nhận, tỷ lệ mở và click theo loại email
Insights	`/insights`	Nhịp hoạt động theo giờ, giá trị đơn hàng, kênh, khách hàng trung thành
Products	`/products`	Doanh thu theo ngành hàng và thiết bị
Geography	`/geography`	Doanh thu ròng theo 13 tỉnh/thành phố trên bản đồ Việt Nam
Data Health	`/operations`	Sức khỏe hệ thống, kiểm chứng chất lượng, lineage, kiểm toán

Phân bổ doanh thu (BR-01)

Trong 20 tháng lịch sử, bốn nền tảng ghi nhận 2.614.669 lượt hiển thị, 227.830 click và tổng chi phí 1.339.776.177 VND. Hình 4.4 đặt cạnh nhau tỷ trọng chi phí và doanh thu gộp được phân bổ cho từng nền tảng. Shopee Ads chỉ chiếm 18,7% chi phí nhưng được phân bổ doanh thu gộp cao nhất (20,54 tỷ VND từ 9.158 đơn), trong khi Google Ads chiếm tỷ trọng chi phí lớn nhất (29,7%) nhưng doanh thu phân bổ chỉ đứng thứ ba. Đây chính là loại phát hiện mà một báo cáo riêng lẻ của từng nền tảng không thể đưa ra, và là cơ sở để CMO xem xét chuyển ngân sách. Màn hình Analysis cho phép so sánh kết quả này giữa ba mô hình First-Touch, Last-Touch và Linear trước khi quyết định.



Hình 4.4: Tỷ trọng chi phí và doanh thu gộp được phân bổ theo nền tảng quảng cáo (01/2025 – 08/2026)

Giám sát, cảnh báo và ngân sách (BR-02 – BR-05)

Trong khi lịch sử dừng ở 31/08/2026, luồng tháng 09/2026 tiếp tục chảy vào màn hình Live View. Để kiểm tra số liệu trực tiếp có đáng tin hay không, nhóm đối chiếu màn hình với file nguồn tại cùng thời điểm phát lại (Bảng 4.2): sau khi khử trùng, lọc đồng thuận và loại phiên bot, hai bên khớp tuyệt đối.

Bảng 4.2: *Đối chiếu Live View với file nguồn tại thời điểm phát lại*

Chỉ số	Live View	File nguồn
Khách truy cập trong 30 phút gần nhất	8	8
Khách truy cập / phiên trong ngày	224 / 231	224 / 231
Đơn hàng	25	25
Doanh thu gộp (VND)	60.324.000	60.324.000
Hoàn tiền (VND)	6.805.000	6.805.000
Doanh thu ròng (VND)	53.519.000	53.519.000

Cảnh báo (BR-03), giải ngân (BR-04) và mô phỏng (BR-05) cố ý dựa trên số liệu đã kiểm chứng của `campaign_performance_daily`, vì chi phí quảng cáo chỉ có ở nhánh lô. Ba loại quy tắc được hỗ trợ là vượt ngân sách, chi phí tăng đột biến so với ngày trước và tỷ trọng hoàn tiền; mỗi lần cảnh báo lưu kèm toán hạng, ngưỡng, cửa sổ thời gian và mã release làm bằng chứng. Phép mô phỏng tính doanh thu ròng dự kiến theo ROAS ròng lịch sử của từng chiến dịch và ghi rõ giả định "kịch bản tuyến tính, không phải dự báo".

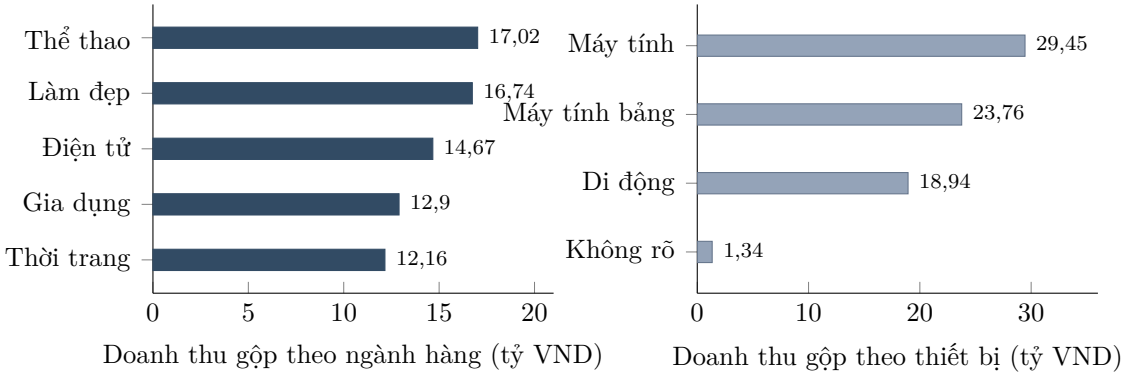
Kiểm chứng và sức khỏe hệ thống (BR-06)

Màn hình Data Health cho quản trị viên thấy release đang phục vụ, lineage từ snapshot Bronze đến `proof_digest`, kết quả kiểm chứng lại toàn bộ bảng và đối soát tài chính, cùng nút chạy thử nghiệm làm hỏng có kiểm soát đã mô tả ở Ví dụ 4. Trên release story-demo, tổng doanh thu gộp 73.488.628.000 VND và hoàn tiền 4.329.091.000 VND khớp nhau giữa `commerce_daily` và `campaign_performance_daily`, cho doanh thu ròng 69.159.537.000 VND trên 32.774 đơn hàng.

Phễu, Customer 360 và cohort (BR-07 – BR-09)

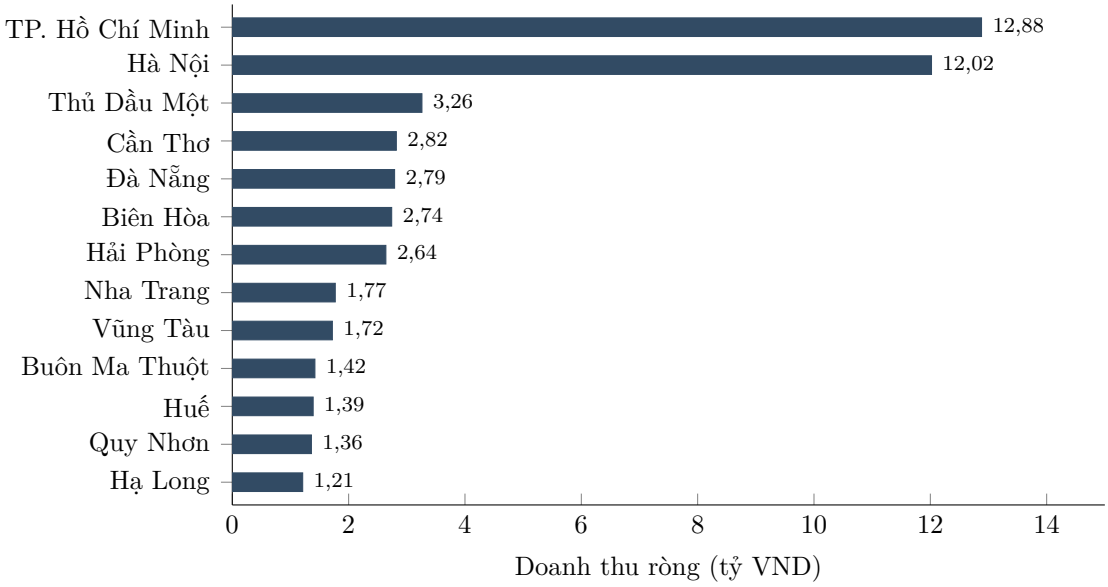
Lớp hành trình gồm 298.032 phiên web/app không phải bot. Kênh web có 198.995 phiên với tỷ lệ chuyển đổi 11,9%, ứng dụng có 99.037 phiên với 8,5%. Lưu lượng từ quảng cáo trả phí chiếm phần lớn (219.770 phiên xem trang), phần còn lại đến từ tìm kiếm tự nhiên, truy cập trực tiếp và email, chiếm 26,3% số phiên sau quản trị. Nhịp hoạt động theo giờ Việt Nam cho thấy khung 20:00–22:00 có số phiên gấp 12,5 lần khung 02:00–04:00, cao điểm vào 21:00 thứ Bảy. Đây là thông tin hữu ích để lên lịch gửi email và chạy quảng cáo. Giá trị đơn hàng có trung vị 1.797.000 VND và phân vị 90 là 4.497.000 VND.

Ở góc độ sản phẩm (Hình 4.5), Thể thao và Làm đẹp dẫn đầu doanh thu gộp; máy tính để bàn và máy tính bảng tạo ra phần lớn doanh thu dù lưu lượng di động không nhỏ. Màn hình Customers cung cấp 748 dòng cohort theo tháng (các cohort từ 2023 đến 2026), hồ sơ Customer 360 với dòng thời gian hoạt động, phân đoạn RFM có lưu phiên bản và danh sách cơ hội như khách có nguy cơ rời bỏ (đã mua nhưng không quay lại từ 30 ngày) hoặc khách bỏ giỏ hàng.



Hình 4.5: Doanh thu gộp đã kiểm chứng theo ngành hàng và theo thiết bị mua hàng

Về địa lý (Hình 4.6), TP. Hồ Chí Minh và Hà Nội chiếm lần lượt 26,8% và 25,0% doanh thu ròng đã xác định vị trí. Khoảng 21,17 tỷ VND đến từ hồ sơ ẩn danh hoặc chưa khớp được với khách hàng nên được tách riêng thành "không xác định" thay vì gán ngẫu nhiên cho một tỉnh thành; tổng các phần vẫn bằng đúng doanh thu ròng của Gold.



Hình 4.6: Doanh thu ròng đã kiểm chứng theo 13 tỉnh/thành phố (khách hàng đã xác định vị trí)

Quyền riêng tư và tính lại lịch sử (BR-10)

Quản trị dữ liệu thể hiện trực tiếp trên số liệu: 1.084 đơn hàng của khách từ chối đồng thuận phân tích không xuất hiện trong bất kỳ màn hình đã kiểm chứng nào. Hồ sơ trên

giao diện chỉ hiện bút danh `profile_ref`; phân đoạn chỉ được đánh dấu có thể kích hoạt khi khách hàng có đồng thuận marketing. Người dùng `viewer` không thấy và không gọi được các thao tác chi tiết hay vận hành; mọi quyết định truy cập nằm trong nhật ký kiểm toán. Việc tính lại lịch sử chạy cô lập qua `historical_rebuild_job` (Ví dụ 5) và không thay đổi release đang phục vụ.

Tổng hợp đối chiếu yêu cầu

Bảng 4.3 truy vết mỗi yêu cầu nghiệp vụ tới màn hình, API và dữ liệu đã hiện thực.

Bảng 4.3: *Truy vết yêu cầu nghiệp vụ tới màn hình, API (tiền tố `/api/v1`) và dữ liệu*

BR	Màn hình	API chính	Dữ liệu

BR-01	Analysis
	Ads
		`GET /analysis/attribution`
		`GET /analysis/journeys`
			`attribution_daily`
			`campaign_performance_daily`
			`paid_touches`

BR-02	Overview Live	`GET /dashboard` `GET /live/view`	`marketpulse_live.*`
BR-03	Overview	`GET/POST /alert-rules` `POST /alerts/evaluate`	`campaign_performance_daily` bảng cảnh báo PostgreSQL
BR-04	Overview	`GET /budget-pacing/{campaign_id}`	`campaign_settings` `campaign_performance_daily`
BR-05	Overview	`POST /budget-simulation`	`campaign_performance_daily`
BR-06	Data Health	`POST /ops/quality/check` `POST /ops/quality/probe`	release, `certified_bundles` `quality_reports`
BR-07	Analysis	`GET /analysis/funnel`	`funnel_daily`

BR-08	Customers
	Email
	Insights
		`GET /lifecycle/profiles`
		`GET /lifecycle/segments`
		`GET /lifecycle/email-engagement`
		`GET /insights/activity`
			`customer_360`
			`mp_journey_*.events`
			`segment_revisions`

BR-09	Customers
	Products
	Geography
		`GET /lifecycle/cohorts`
		`GET /insights/products`
		`GET /insights/geography`
			`cohort_retention`
			`commerce_daily`
			`customers`

BR-10	Toàn bộ	`POST /auth/login` `PUT /ops/users/{id}`	`auth_*`, `access_audit` Silver đã quản trị

Chương 5

Đánh giá thực nghiệm

Đề bài yêu cầu đánh giá giải pháp trên ba mặt: tính đúng đắn của dữ liệu sau xử lý, hiệu năng của giải pháp Kỹ thuật Dữ liệu, và hiệu quả hỗ trợ khai thác dữ liệu qua ứng dụng. Chương này định nghĩa trước các độ đo, sau đó trình bày kết quả của từng mặt. Mọi tiêu chí đều được định nghĩa bằng công thức hoặc bằng phép so sánh chính xác với một oracle độc lập.

5.1 Môi trường và độ đo

Cấu hình môi trường thực nghiệm

Các thực nghiệm hiệu năng chạy trong một dự án Docker Compose cô lập (`compose.evaluation.yaml`) trên một máy với 15 CPU, 15,60 GiB bộ nhớ khả dụng, kiến trúc aarch64 và Docker Engine 29.4.0. Thực nghiệm xử lý luồng dùng Kafka 4.3.1 và Java 17, mỗi làn một partition, chu kỳ commit/checkpoint 1 giây cho cả hai động cơ. Thực nghiệm phục vụ phân tích giới hạn mỗi container CSDL ở 1,5 GiB bộ nhớ, tắt bộ nhớ đệm kết quả của ClickHouse, và chạy 20 lượt khởi động (warm-up) trước 100 lượt đo tuần tự cho mỗi truy vấn. Mỗi lượt đo luồng tạo một job hoặc ứng dụng mới. Các thực nghiệm hiệu năng là phép đo một lần trên máy cục bộ dùng chung, nên chỉ dùng để so sánh có kiểm soát giữa các phương án, không phải để suy ra năng lực sản xuất.

Định nghĩa các độ đo

Với N bản ghi đầu vào, t_i^{send} là thời điểm bản ghi i được gửi và t_i^{obs} là thời điểm kết quả tương ứng được quan sát thấy *đã commit*:

- **Thông lượng** (bản ghi/giây): $TP = \dfrac{N}{\max_i t_i^{\text{obs}} - \min_i t_i^{\text{send}}}$.
- **Độ trễ commit** của bản ghi i: $\ell_i = t_i^{\text{obs}} - t_i^{\text{send}}$, bao gồm thời gian xếp hàng và thời gian hiển thị giao dịch. Phân vị p tính theo phương pháp *nearest-rank*: $P_p = \ell_{(\lceil p \cdot n/100 \rceil)}$ trên dãy ℓ đã sắp xếp tăng dần; báo cáo P_{50}, P_{95}, P_{99}.

- **Thời gian phục hồi**: $T_{\text{rec}} = t_{\text{out}} - t_{\text{fail}}$, trong đó t_{fail} là lúc gây sự cố và t_{out} là lúc đầu ra mới đầu tiên được commit.
- **Thời gian CPU**: tổng mức tăng của bộ đếm CPU của các tiến trình liên quan trong cửa sổ đo; **bộ nhớ đỉnh**: giá trị lớn nhất của bộ nhớ làm việc được lấy mẫu.
- **Độ trễ truy vấn / API**: thời gian thực của một truy vấn hoặc một request HTTP có xác thực, báo cáo P_{50}, P_{95}, P_{99} trên 100 mẫu.
- **Hệ số tăng tốc**: $S = P_{95}^{\text{baseline}} / P_{95}^{\text{candidate}}$.
- **Tính đúng đắn**: một phép kiểm tra *đạt* khi giá trị của hệ thống bằng *chính xác* giá trị của oracle độc lập. Số tiền là số nguyên nên sai số cho phép bằng 0.

5.2 Tính đúng đắn của dữ liệu

Hệ thống kiểm thử tự động

Kho mã có 683 hàm kiểm thử Python, 35 kiểm thử Java cho job Flink, 4 kiểm thử Java cho job đánh giá, 31 kiểm thử dữ liệu dbt và 20 file đặc tả kiểm thử giao diện bằng Playwright. Kiểm thử bao phủ toàn bộ đường ống: tính tất định của dữ liệu sinh, hợp đồng Kafka, thứ tự và tốc độ phát lại, khử trùng lặp, sự kiện trễ, phiên, chỉ số chiến dịch, quá trình nạp Bronze, quản trị Silver, các mô hình Gold, công bố và toàn bộ API. Ngoài kiểm thử đơn vị, các kịch bản Docker thật kiểm tra phục hồi sau sự cố, khởi động lại catalog/lưu trữ và dựng lại lịch sử cô lập.

Đối soát độc lập trên story-demo

Script `story_demo_oracle.py` tính lại các chỉ số từ chính file nguồn của bộ sinh (không qua đường ống) rồi so với release đã kiểm chứng trên ClickHouse. Kết quả: **386/386 phép kiểm tra bằng nhau** (Bảng 5.1). Chênh lệch duy nhất giữa dữ liệu thô và dữ liệu đã kiểm chứng được giải thích trọn vẹn bởi quản trị: 33.858 đơn hàng thô so với 32.774 đơn đã kiểm chứng, chênh đúng 1.084 đơn hàng không có đồng thuận phân tích.

Bảng 5.1: *Kết quả đối soát tính đúng đắn của dữ liệu*

Tập dữ liệu	Phép kiểm tra	Kết quả
Fixture	Doanh thu gộp 27.160.000, hoàn 4.967.000, ròng 22.193.000 VND bảo toàn qua cả ba mô hình phân bổ	Khớp tuyệt đối
Story-demo	Hiển thị, click, chi phí của 4 nền tảng × 20 tháng (240 phép kiểm tra)	240/240
Story-demo	Đơn hàng, doanh thu gộp, hoàn tiền theo tháng; doanh thu phân bổ theo nền tảng; doanh thu theo 13 tỉnh/thành và nhóm ẩn danh	Khớp, tổng cộng 386/386
Story-demo (Silver)	Email theo loại × tháng; phiên theo nguồn truy cập sau quản trị	60/60, khớp
Live View	Khách truy cập, phiên, đơn hàng, doanh thu so với file nguồn tại cùng thời điểm phát lại	Khớp tuyệt đối
Evaluation 10k/100k	Mỗi đầu ra đã commit so với oracle Python; mã băm chuẩn hóa đầu vào/đầu ra giữa Flink và Kafka Streams	Khớp ở cả hai quy mô

Đối chiếu đầu ra Flink và Kafka Streams

Trước khi so sánh tốc độ, hai động cơ phải cho cùng một kết quả. Khối lượng công việc tài chính tất định gồm cả bản ghi trùng, bản ghi sai giá trị, bản ghi trễ trong ngưỡng và trễ quá ngưỡng. Với mỗi đầu ra đã commit, một oracle Python độc lập kiểm tra tính đầy đủ của chuỗi, cách phân loại và số tiền trung gian theo từng chiến dịch. Kết quả xác nhận: bản ghi trùng không làm thay đổi tổng, giá trị sai không làm sai lệch số tiền, bản ghi trễ trong ngưỡng được tính và bản ghi trễ quá ngưỡng được ghi nhận rồi loại ra. Mã băm chuẩn hóa của đầu vào và đầu ra giống nhau giữa hai động cơ.

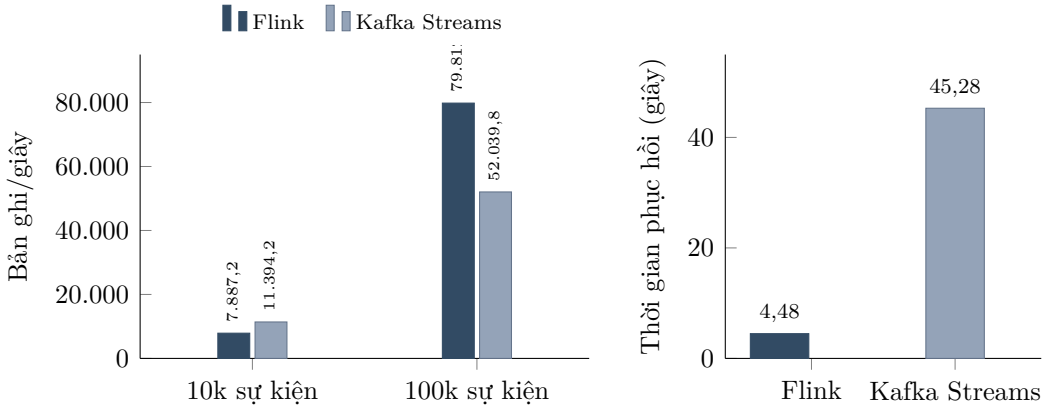
5.3 Flink và Kafka Streams

Thông lượng và độ trễ

Bảng 5.2 và Hình 5.1 cho thấy kết quả phụ thuộc rõ vào quy mô. Ở `eval-medium` (100.000 sự kiện), Flink đạt 79.812,6 bản ghi/giây, gấp 1,53 lần Kafka Streams (52.039,8), và độ trễ P_{95} thấp hơn 33,6% (1.094,34 ms so với 1.648,05 ms). Ngược lại, ở `eval-small` (10.000 sự kiện), Kafka Streams nhanh hơn với 11.394,2 bản ghi/giây và P_{95} 858,45 ms so với 7.887,2 bản ghi/giây và 1.240,90 ms của Flink. Đây là kết quả hợp lý: Kafka Streams là thư viện nhúng nên khởi động nhẹ, phù hợp với tải nhỏ; khi tải tăng, động cơ phân tán của Flink mới phát huy lợi thế. Độ trễ ở đây bao gồm cả thời gian hiển thị giao dịch với chu kỳ commit 1 giây, nên không phải độ trễ từ sự kiện đến bảng điều khiển trong vận hành thực tế.

Bảng 5.2: *Thông lượng và độ trễ commit của Apache Flink 2.2.1 và Kafka Streams 4.3.1*

Động cơ	Quy mô	Bản ghi/giây	P_{50} (ms)	P_{95} (ms)	P_{99} (ms)
Flink	eval-small (10k)	7.887,2	1.133,39	1.240,90	1.242,23
Kafka Streams	eval-small (10k)	11.394,2	852,90	858,45	858,76
Flink	eval-medium (100k)	79.812,6	1.085,17	1.094,34	1.096,43
Kafka Streams	eval-medium (100k)	52.039,8	1.635,03	1.648,05	1.651,72



Hình 5.1: Thông lượng theo quy mô và thời gian phục hồi sau sự cố của Flink và Kafka Streams

Thời gian phục hồi sau sự cố

Khi tiến trình xử lý bị dừng đột ngột ở quy mô `eval-small`, Flink khôi phục từ checkpoint số 4 và có đầu ra mới được commit sau 4,476 giây. Kafka Streams giữ nguyên application ID và trạng thái cục bộ nhưng phải phát lại changelog của trạng thái, mất 45,277 giây, chậm hơn khoảng 10,1 lần. Với nhánh luồng của MarketPulse 360, nơi trạng thái gồm khử trùng lặp 7 ngày, phiên và chỉ số chiến dịch, khả năng phục hồi nhanh là yếu tố quyết định. Kết hợp với lợi thế ở tải lớn và các tính năng thời gian sự kiện, kết quả củng cố lựa chọn Flink làm động cơ chính. Cần lưu ý phạm vi so sánh: đây là một bộ tổng hợp tài chính giới hạn, chưa bao gồm gom phiên, toàn bộ xử lý hợp đồng và TTL trạng thái của job thật.

5.4 ClickHouse và PostgreSQL

Độ trễ truy vấn theo quy mô

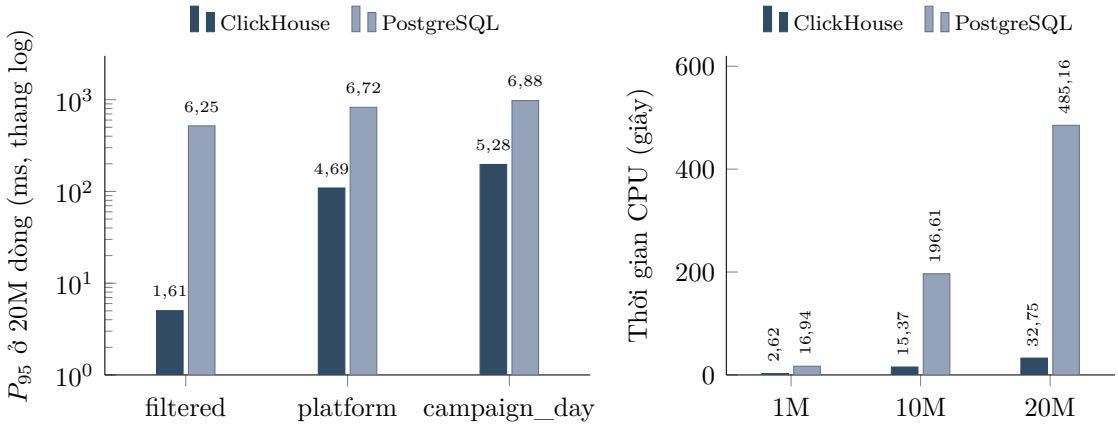
Hai hệ thống được nạp cùng bảng sự kiện dạng Gold với tiền là số nguyên, ở ba quy mô 1, 10 và 20 triệu dòng, và chạy ba truy vấn đại diện: tổng theo nền tảng (`platform`), tổng theo chiến dịch và ngày (`campaign_day`), và truy vấn lọc theo chiến dịch trong một khoảng ngày (`filtered`). PostgreSQL được tạo chỉ mục kép `(campaign, event_day)`;

ClickHouse sắp xếp dữ liệu theo `(campaign, event_day, row_id)`. Mọi lượt đo đều trả về đúng đáp án của oracle.

Bảng 5.3: *Độ trễ truy vấn (ms) của ClickHouse 26.8 và PostgreSQL 17.6*

		ClickHouse			PostgreSQL		
Quy mô	**Truy vấn**	P_{50}	P_{95}	P_{99}	P_{50}	P_{95}	P_{99}
1M	filtered	1,93	2,86	3,11	2,53	3,28	3,87
1M	platform	7,13	11,04	12,22	32,03	38,43	49,69
1M	campaign_day	10,98	16,12	18,08	37,06	43,22	45,25
10M	filtered	3,60	4,47	5,22	88,86	109,82	158,55
10M	platform	47,88	53,59	54,39	333,15	422,23	617,17
10M	campaign_day	71,06	82,23	86,73	389,00	518,30	583,10
20M	filtered	4,28	5,02	5,29	378,42	518,54	651,49
20M	platform	98,07	109,05	122,08	740,74	826,55	1.074,45
20M	campaign_day	148,52	196,89	221,05	849,06	977,09	1.298,05

Ở 1 triệu dòng, hai hệ thống gần như ngang nhau với truy vấn có chỉ mục (2,86 ms so với 3,28 ms). Khoảng cách nới rộng nhanh khi dữ liệu tăng (Hình 5.2): ở 20 triệu dòng, ClickHouse nhanh hơn 103,3 lần với `filtered` (5,02 ms so với 518,54 ms), 7,58 lần với `platform` và 4,96 lần với `campaign_day` xét theo P_{95}. Nguyên nhân nằm ở cách lưu trữ: ClickHouse chỉ đọc các cột cần thiết và bỏ qua các khối dữ liệu không thỏa điều kiện nhờ khóa sắp xếp, trong khi PostgreSQL phải đọc từng dòng đầy đủ.



Hình 5.2: Độ trễ truy vấn P_{95} ở 20 triệu dòng và thời gian CPU theo quy mô của ClickHouse và PostgreSQL

Mức tiêu thụ tài nguyên

Trong cùng khối lượng truy vấn, PostgreSQL tiêu tốn 485,16 giây CPU ở 20 triệu dòng, so với 32,75 giây của ClickHouse, tức tiết kiệm khoảng 14,8 lần. Đổi lại, ClickHouse dùng nhiều bộ nhớ hơn (đỉnh khoảng 1 GiB so với 534,2 MiB của PostgreSQL) do xử lý theo khối cột lớn. Kết quả xác nhận cách phân công trong kiến trúc: ClickHouse cho truy

vấn phân tích, PostgreSQL cho dữ liệu vận hành cần ràng buộc và giao dịch. Phương án Google BigQuery/BigLake không được đo nên không có kết luận về phương án này.

5.5 Hiệu quả của ứng dụng

Kết quả các tác vụ ra quyết định

Hiệu quả của ứng dụng được đánh giá qua mười tác vụ ra quyết định, mỗi tác vụ gắn với một bên liên quan và một bằng chứng kiểm chứng độc lập (Bảng 5.4). Một kịch bản tự động đăng nhập với vai trò `operator`, thực hiện từng tác vụ qua API và đối chiếu kết quả với oracle. Cả mười tác vụ đều đạt, và cấu hình được so sánh trước–sau để bảo đảm không ngân sách, quy tắc, phân đoạn hay bản phát hành nào bị thay đổi ngoài ý muốn.

Bảng 5.4: *Mười tác vụ ra quyết định và kết quả kiểm chứng tự động*

Mã	Bên liên quan	Tác vụ và quyết định cần đưa ra	Kết quả
DT-01	Performance	So sánh KPI trực tiếp với doanh thu ròng đã kiểm chứng, chọn chiến dịch cần xem	Đạt
DT-02	Performance	Giải thích một cảnh báo vượt ngân sách từ ngưỡng và bằng chứng đã lưu	Đạt
DT-03	CMO	Đọc tiến độ giải ngân của chiến dịch trong kỳ cấu hình	Đạt
DT-04	CMO	Mô phỏng phân bổ ngân sách, hiểu giả định tuyến tính	Đạt
DT-05	Product	Xác định bước rơi rụng lớn trong phễu	Đạt
DT-06	Analyst	So sánh ba mô hình phân bổ, kiểm tra bảo toàn doanh thu	Đạt
DT-07	CRM	Xem dòng thời gian hồ sơ đã ẩn danh	Đạt
DT-08	CRM	Xem trước và lưu phân đoạn, chỉ kích hoạt khách có đồng thuận	Đạt
DT-09	CRM	Giải thích quần thể cohort và mẫu số hoạt động	Đạt
DT-10	CRM	Phân biệt khách giá trị cao với quyền được kích hoạt	Đạt

Thời gian API của kịch bản tự động không đồng nghĩa với thời gian một người dùng thật hoàn thành tác vụ. Nhóm đã chuẩn bị phiếu thử nghiệm với người dùng đại diện cho từng vai trò; phần quan sát này chưa được thực hiện trong phạm vi báo cáo.

Độ trễ của REST API

Bảng 5.5 tổng hợp độ trễ của các API chính, mỗi API đo 100 request HTTP có xác thực sau 20 lượt khởi động. API bảng điều khiển tổng hợp có P_{95} là 54,35 ms; các API phân tích và vòng đời có P_{95} từ 13,64 ms đến 32,39 ms; API minh họa hành trình chậm hơn (77,84 ms) vì phải quét và chuẩn hóa nhiều điểm chạm. Tất cả đều nằm dưới ngưỡng 100 ms, đủ nhanh để người dùng thao tác liên tục mà không cảm thấy chờ.

Bảng 5.5: *Độ trễ phản hồi của các REST API chính (ms)*

API	**Tập dữ liệu**	P_{50}	P_{95}	P_{99} / lớn nhất
Bảng điều khiển tổng hợp	release fixture	37,05	54,35	56,12
Phễu chuyển đổi	fixture	13,85	26,58	41,58
Phân bổ doanh thu	fixture	15,55	18,96	30,67
Hành trình mẫu (5 ví dụ)	fixture	39,17	77,84	96,07
Cohort	fixture	9,53	13,64	18,76
Danh sách hồ sơ	fixture	20,80	28,53	34,63
Dòng thời gian hồ sơ	fixture	19,84	22,00	23,11
Xem trước phân đoạn	fixture	24,80	32,39	33,28

Chương 6

Thách thức và kết luận

6.1 Ba thách thức vận hành

Một hệ thống dữ liệu marketing chạy được trên dữ liệu mẫu chưa đủ để vận hành thực tế. Trong quá trình xây dựng, nhóm gặp ba thách thức mà hầu hết doanh nghiệp bán lẻ đa kênh đều phải đối mặt: dữ liệu bẩn, dữ liệu nhạy cảm, và chi phí tăng theo khối lượng. Bảng 6.1 tóm tắt cách MarketPulse 360 xử lý từng thách thức trên bốn chặng của đường ống; các tiểu mục sau trình bày chi tiết.

Bảng 6.1: *Ba thách thức vận hành và giải pháp trên từng chặng của đường ống*

Chặng	Dữ liệu bẩn	Quyền riêng tư	Chi phí và mở rộng
Thu thập	Hợp đồng sự kiện có phiên bản trên Apicurio; bản ghi sai hợp đồng vào `dlq.validation` kèm lý do	Chỉ thu nhận trường cần cho phân tích; trạng thái đồng thuận đi kèm hồ sơ CRM	Retention theo hợp đồng: 7 ngày cho raw/clean, 30 ngày compact cho Gold
Xử lý	Flink khử trùng lặp theo khóa sự kiện (TTL 7 ngày), xử lý trễ bằng watermark; bản ghi lỗi vào `dlq.flink-validation` kèm mã lỗi	Silver loại sự kiện bị từ chối đồng thuận và phiên bot trước khi tính toán	Trạng thái có TTL; song song hóa theo partition Kafka
Lưu trữ	Silver đối soát hai chiều raw–clean–DLQ, phát hiện xung đột payload; Gold kiểm thử bằng dbt	Hồ sơ chỉ xuất hiện qua `profile_ref` băm theo release; mật khẩu băm scrypt	Iceberg trên MinIO, snapshot bất biến; ClickHouse lưu cột, nén tốt
Phân phối	Chỉ công bố release đã qua 7 bước chứng thực; release lỗi không bao giờ được kích hoạt	RBAC ba vai trò, phiên mờ có hạn, nhật ký kiểm toán đã làm sạch; kích hoạt chỉ khi đủ đồng thuận	API ghim release, truy vấn đọc ít cột; mở rộng ngang theo partition

Dữ liệu bẩn (Dirty Data)

Dữ liệu marketing thực tế luôn có lỗi: sự kiện gửi trùng do client thử lại, sự kiện đến trễ vì thiết bị mất mạng, giá trị sai kiểu hoặc thiếu khóa, và cùng một khóa nhưng payload

khác nhau. Bộ sinh dữ liệu của dự án cố ý tạo ra các lỗi này để kiểm chứng cách hệ thống phản ứng.

Hệ thống chặn lỗi theo nhiều lớp thay vì dồn về một chỗ. Ở cửa vào, mỗi sự kiện được kiểm tra theo hợp đồng có phiên bản; bản ghi không hợp lệ được chuyển sang `marketing.dlq.validation.v1` kèm lý do thay vì bị bỏ đi âm thầm. Trong Flink, khử trùng lặp theo khóa sự kiện với TTL 7 ngày bảo đảm sự kiện gửi lại không làm tăng doanh thu hai lần; watermark 5 phút và cửa sổ hiệu chỉnh 10 phút cho phép tính sự kiện trễ trong ngưỡng, còn sự kiện trễ quá ngưỡng được ghi nhận và loại ra một cách tường minh. Ở tầng Silver, Spark đối soát hai chiều giữa dữ liệu thô, dữ liệu sạch và DLQ để mọi bản ghi đều có một kết cục được giải thích, đồng thời so sánh mã băm logic để phát hiện các bản ghi trùng khóa nhưng khác nội dung. Cuối cùng, 31 kiểm thử dbt và bước kiểm tra phương trình tài chính chặn mọi sai lệch trước khi dữ liệu đến người dùng.

Hiệu quả của cách tiếp cận này được thể hiện ở Chương 5: trên tập story-demo, 386 phép đối soát độc lập đều khớp tuyệt đối, và chênh lệch giữa 33.858 đơn hàng thô với 32.774 đơn đã kiểm chứng được giải thích trọn vẹn bởi quy tắc đồng thuận.

Dữ liệu nhạy cảm và quyền riêng tư

Hồ sơ khách hàng, lịch sử mua hàng và hành vi duyệt web là dữ liệu cá nhân. Tại Việt Nam, Nghị định 13/2023/NĐ-CP [21] và Luật Bảo vệ dữ liệu cá nhân số 91/2025/QH15 [22] yêu cầu xử lý dữ liệu cá nhân phải có căn cứ, đúng mục đích và có biện pháp bảo vệ phù hợp. MarketPulse 360 hiện thực các biện pháp kỹ thuật sau:

- **Đồng thuận là điều kiện, không phải chú thích.** Sự kiện của khách từ chối đồng thuận phân tích bị loại ở Silver trước khi đi vào bất kỳ chỉ số nào. Một khách hàng chỉ được đưa vào danh sách kích hoạt khi đã định danh trong CRM và có *cả* đồng thuận phân tích lẫn đồng thuận marketing hiện hành; trạng thái không rõ hoặc bị từ chối đều được xử lý theo hướng an toàn (*fail closed*). Khi khách từ chối đồng thuận phân tích, chi tiết dòng thời gian của họ bị ẩn.

- **Giảm khả năng nhận diện.** Giao diện vòng đời khách hàng không dùng khóa gốc mà dùng `profile_ref`, là giá trị SHA-256 của định danh release ghép với khóa hồ sơ. Vì giá trị này gắn với từng release, không thể dùng nó để nối dữ liệu giữa các release khác nhau.

- **Bảo vệ truy cập.** Mật khẩu được băm bằng scrypt có muối; token phiên chỉ được lưu dưới dạng băm SHA-256 và có thời hạn. Ba vai trò `viewer`, `operator` và `admin` giới hạn chức năng theo nguyên tắc quyền tối thiểu, và mọi thao tác quản trị được ghi vào nhật ký kiểm toán chỉ-ghi-thêm đã làm sạch.

- **Tách bạch dữ liệu.** Tài khoản, bằng chứng chất lượng và siêu dữ liệu xác thực nằm trong PostgreSQL vận hành; ClickHouse chỉ chứa dữ liệu phân tích. Chính

sách retention chỉ xóa phiên đăng nhập và chuỗi số liệu giám sát, không động đến dữ liệu đã chứng thực.

Cần nói rõ giới hạn hiện tại: hệ thống chưa mã hóa hay băm các trường liên hệ như email, số điện thoại ở tầng lưu trữ, và chưa có quy trình tự động đáp ứng yêu cầu xóa dữ liệu của chủ thể. Đây là các hạng mục cần bổ sung trước khi triển khai với dữ liệu thật.

Chi phí và khả năng mở rộng

Chi phí của một hệ thống dữ liệu tăng theo ba hướng: lưu trữ, tính toán và vận hành. MarketPulse 360 kiểm soát cả ba bằng các quyết định thiết kế cụ thể.

Về lưu trữ, retention của Kafka được khai báo trong hợp đồng: 7 ngày cho topic raw và clean, 30 ngày kèm nén log (*compaction*) cho topic Gold, nên dữ liệu trên broker không phình ra vô hạn. Lịch sử dài hạn nằm trên Iceberg với lưu trữ đối tượng MinIO, rẻ hơn nhiều so với đĩa của CSDL và có thể thay bằng S3 hoặc GCS mà không phải đổi định dạng bảng. Về tính toán, thực nghiệm ở Mục 5.4 cho thấy ClickHouse dùng ít hơn khoảng 14,8 lần thời gian CPU so với PostgreSQL ở 20 triệu dòng. Đây là chênh lệch trực tiếp về chi phí máy chủ khi khối lượng truy vấn tăng. Về vận hành, toàn bộ ngăn xếp là phần mềm mã nguồn mở, chạy bằng Docker Compose và được điều phối bởi Dagster, nên không phát sinh phí bản quyền và có thể tái lập trên bất kỳ máy nào.

Về khả năng mở rộng, các thành phần đều mở rộng ngang được: topic Kafka có 6 partition (3 với topic Gold), job Flink chạy song song theo partition, Spark xử lý theo snapshot Iceberg, còn ClickHouse có thể thêm shard khi dữ liệu lớn hơn. Giới hạn hiện tại là chứng thực đang chạy lại toàn bộ release mỗi đêm; khi dữ liệu tăng đáng kể, cần chuyển sang chứng thực nối tiếp như trình bày ở Mục 6.3.

6.2 Kết luận

Báo cáo đã trình bày MarketPulse 360, một hệ thống Kỹ thuật Dữ liệu hoàn chỉnh cho phân tích marketing đa kênh, đi từ yêu cầu nghiệp vụ đến ứng dụng ra quyết định. Các đóng góp chính gồm:

- **Kiến trúc kết hợp luồng và lô có ranh giới rõ ràng.** Kafka, Apicurio và Flink cung cấp chỉ số gần thời gian thực với ngữ nghĩa *exactly-once*; Spark, Iceberg, Nessie và dbt xây dựng các tầng Bronze–Silver–Gold có thể tái lập. Ứng dụng luôn hiển thị tách biệt số liệu trực tiếp và số liệu đã chứng thực, nên người dùng biết mình đang đọc loại số nào.
- **Cơ chế chứng thực trước khi công bố.** Mỗi release phải qua bảy bước kiểm tra, từ ghim snapshot, chạy kiểm thử dbt, kiểm tra phương trình tài chính đến so khớp checksum, trước khi được kích hoạt nguyên tử qua `publication_journal`. Các thí

nghiệm cố ý làm hỏng dữ liệu cho thấy release lỗi bị chặn và release đang hoạt động không bị ảnh hưởng.

- **Đúng đắn được chứng minh bằng đối soát độc lập.** 386/386 phép kiểm tra trên story-demo khớp tuyệt đối với oracle tính từ dữ liệu nguồn; Flink và Kafka Streams cho đầu ra giống hệt nhau ở cả hai quy mô thử nghiệm.
- **Lựa chọn công nghệ dựa trên đo đạc.** Flink nhanh hơn 1,53 lần về thông lượng ở 100.000 sự kiện và phục hồi nhanh hơn khoảng 10 lần; ClickHouse nhanh hơn tới 103 lần về P_{95} và tiết kiệm khoảng 14,8 lần CPU so với PostgreSQL ở 20 triệu dòng.
- **Ứng dụng phục vụ ra quyết định.** Mười không gian làm việc và 41 route API đáp ứng mười yêu cầu nghiệp vụ; mười tác vụ ra quyết định đều đạt trong kiểm chứng tự động, với độ trễ API P_{95} dưới 100 ms.

Bên cạnh đó, nhóm ghi nhận các giới hạn: số liệu hiệu năng là phép đo một lần trên máy cục bộ; đánh giá với người dùng thật chưa được thực hiện; phương án BigQuery/BigLake chưa được đo; và dữ liệu sử dụng là dữ liệu mô phỏng. Các giới hạn này không làm thay đổi kết luận về kiến trúc, nhưng cần được bổ sung trước khi đưa hệ thống vào môi trường thực tế.

6.3 Hướng phát triển

Dựa trên các giới hạn nêu trên, nhóm đề xuất bốn hướng phát triển tiếp theo:

1. **Chứng thực nối tiếp.** Thay vì dựng lại và kiểm tra toàn bộ release mỗi đêm, chỉ xử lý các partition có dữ liệu thay đổi và tái sử dụng bằng chứng của các partition không đổi. Hướng này giảm chi phí tính toán khi dữ liệu lịch sử tăng mà vẫn giữ nguyên bảo đảm về tính đúng đắn.
2. **Mở rộng thực nghiệm.** Lặp lại các thực nghiệm nhiều lần để báo cáo khoảng tin cậy, đo trên cụm nhiều nút, và bổ sung phương án Google BigQuery/BigLake để so sánh chi phí–hiệu năng với ClickHouse tự vận hành.
3. **Mô hình dự báo trên Customer 360.** Xây dựng mô hình dự báo khả năng rời bỏ (*churn*) và giá trị vòng đời khách hàng (CLV) dựa trên các đặc trưng RFM và cohort đã có, để chuyển từ phân tích mô tả sang phân tích dự báo.
4. **Hoàn thiện quyền riêng tư và đánh giá người dùng.** Mã hóa các trường liên hệ, tự động hóa yêu cầu truy cập và xóa dữ liệu của chủ thể theo Luật 91/2025/QH15, và thực hiện thử nghiệm với người dùng đại diện cho từng vai trò theo phiếu đã chuẩn bị.

Tài liệu tham khảo

[1] Nhóm 09 – CO5173, "MarketPulse 360: mã nguồn, dữ liệu mô phỏng và bằng chứng đánh giá." `https://github.com/huyle1901/marketpulse-360`, 2026. Commit dbe614b, 03/11/2026.

[2] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," in *Proceedings of the NetDB Workshop*, 2011.

[3] Apache Software Foundation, "Apache Kafka Documentation." `https://kafka.apache.org/documentation/`, 2026.

[4] Apicurio, "Apicurio Registry Documentation." `https://www.apicur.io/registry/`, 2026.

[5] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, "Apache Flink: Stream and Batch Processing in a Single Engine," *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 38, no. 4, pp. 28–38, 2015.

[6] Apache Software Foundation, "Apache Flink Documentation." `https://nightlies.apache.org/flink/flink-docs-stable/`, 2026.

[7] P. Carbone, S. Ewen, G. Fóra, S. Haridi, S. Richter, and K. Tzoumas, "State Management in Apache Flink: Consistent Stateful Distributed Stream Processing," *Proceedings of the VLDB Endowment*, vol. 10, no. 12, pp. 1718–1729, 2017.

[8] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle, "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing," *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1792–1803, 2015.

[9] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," in *Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR)*, 2021.

[10] MinIO, "MinIO Object Storage Documentation." `https://min.io/docs/`, 2026.

[11] Apache Software Foundation, "Apache Iceberg Table Spec." `https://iceberg.apache.org/spec/`, 2026.

[12] Project Nessie, "Project Nessie: Transactional Catalog for Data Lakes." `https://projectnessie.org/`, 2026.

[13] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," in *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 15–28, 2012.

[14] Apache Software Foundation, "Apache Spark Documentation." `https://spark.apache.org/docs/latest/`, 2026.

[15] Dagster Labs, "Dagster Documentation: Software-Defined Assets." `https://docs.dagster.io/`, 2026.

[16] dbt Labs, "dbt Documentation." `https://docs.getdbt.com/`, 2026.

[17] ClickHouse, Inc., "ClickHouse Documentation: MergeTree Table Engines." `https://clickhouse.com/docs`, 2026.

[18] Vercel, "Next.js Documentation." `https://nextjs.org/docs`, 2026.

[19] S. Ramírez, "FastAPI Documentation." `https://fastapi.tiangolo.com/`, 2026.

[20] The PostgreSQL Global Development Group, "PostgreSQL 17 Documentation." `https://www.postgresql.org/docs/17/`, 2026.

[21] Chính phủ nước Cộng hòa Xã hội Chủ nghĩa Việt Nam, "Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân," 2023.

[22] Quốc hội nước Cộng hòa Xã hội Chủ nghĩa Việt Nam, "Luật Bảo vệ dữ liệu cá nhân, số 91/2025/QH15," 2025.